

Software Testing Report

Progetto: Apache Syncope
Repository GitHub: <https://github.com/valejin/syncope>

Valentina Jin – Matricola: 0365400
Studentessa del Corso di Laurea Magistrale in Ingegneria Informatica
Università degli Studi di Roma Tor Vergata
Anno Accademico 2025/2026

Sommario

1.	Introduzione.....	2
2.	Metodologia	2
3.	Classe: CommandLogic.....	2
3.1	Il SUT: responsabilità e collaboratori.....	2
3.2	Category Partition e Boundary Value Analysis.....	3
3.3	Esecuzione dei test manuali Black-Box.....	4
3.4	Raffinamento strutturale guidato da JaCoCo	4
3.5	Adeguatezza semantica: Mutation Testing (PIT)	4
3.6	Generazione automatica – Randoop.....	5
3.7	Generazione automatica – LLM (Zero-shot, Few-shot, Guided ToT)	5
3.8	EvoSuite – N/A.....	7
3.9	Evoluzione delle varianti C1–C4	7
3.10	Stima di reliability	8
4.	Classe: NotificationDataBinderImpl	8
4.1	Il SUT: responsabilità e collaboratori.....	8
4.2	Category Partition e Boundary Value Analysis.....	8
4.3	Esecuzione dei test manuali Black-Box.....	9
4.4	Raffinamento strutturale guidato da JaCoCo	9
4.5	Adeguatezza semantica: Mutation Testing (PIT)	10
4.6	Generazione automatica – Randoop.....	10
4.7	Generazione automatica – LLM (Zero-shot, Few-shot, Guided ToT)	11
4.8	EvoSuite – N/A.....	12
4.9	Evoluzione delle varianti C1–C4	12
4.10	Stima di reliability	13
5.	Threats to Validity	13
6.	Conclusioni	14
	Appendice: Tabelle e Figure di CommandLogic	15
	Appendice: Tabelle e Figure di NotificationDataBinderImpl.....	29
	Appendice: Prompt LLM per generare le classi varianti C1-C4.....	42

1. Introduzione

Il presente report documenta il lavoro svolto nel modulo di Software Testing, applicato al progetto open source **Apache Syncope**, piattaforma Apache per la gestione dell'identità e degli accessi. L'obiettivo del modulo è dimostrare, attraverso un processo esplicito e ripetibile, la consapevolezza critica con cui le suite di test sono state progettate, misurate ed evolute su due classi del progetto: ogni decisione di test design viene qui motivata rispetto al comportamento osservato del sistema.

Le due classi analizzate sono **CommandLogic** (core/idrepo/logic) e **NotificationDataBinderImpl** (core/provisioning-java/data), individuate nell'ambito della procedura di selezione della Milestone 4 di Prof. Falessi di cui questo lavoro condivide il target. Dal punto di vista del testing, le due classi presentano caratteristiche complementari, sfruttate nella progettazione delle suite: *CommandLogic* espone metodi con parametri compositi e quattro punti di diramazione verso collaboratori esterni – ricerca dell'implementazione, costruzione tramite ImplementationManager, validazione Bean Validation, esecuzione – con una dipendenza da metodi static che richiede una strategia di mock dedicata; *NotificationDataBinderImpl* espone invece cinque collaboratori tutti iniettati da costruttore e un dominio di input più esteso (*NotificationTO*, oltre dieci campi), interamente testabile con mock Mockito standard. La prima classe mette quindi alla prova la gestione di dipendenze statiche in ottica black-box, la seconda la gestione di un dominio di input più ampio.

L'ambiente di sviluppo integra una pipeline di **Continuous Integration** su GitHub Actions (**Listato 1**): un job a matrice esegue i test delle due classi in run separate e filtrate (*-Dtest=CommandLogic* / -Dtest=NotificationDataBinderImpl**), con *fail-fast: false* per non bloccare l'esecuzione sulla seconda classe in caso di fallimento sulla prima. Il workflow gira su JDK 25 (Temurin), attivato su push, pull request e manualmente da interfaccia, con i flag Maven già usati in locale più alcuni aggiuntivi per isolare l'esecuzione dai controlli statici del progetto originale.

2. Metodologia

Le due classi seguono lo stesso flusso metodologico: progettazione manuale Black-Box tramite Category Partition e Boundary Value Analysis (Fase 1), raffinamento White-Box guidato da JaCoCo e mutation testing insieme con la verifica dell'adeguatezza semantica tramite PIT (Fase 2), generazione automatica con Randoop e LLM (Fase 3), evoluzione in varianti refactored C1–C4 con relativa validazione di regressione, stima finale della reliability.

Ogni suite discussa in questo report opera a livello di unità: il soggetto sotto test è sempre una singola classe, con tutti i collaboratori sostituiti da mock. Quando un punto del codice risulta eseguibile solo dal corpo reale di un collaboratore mockato, la riga viene dichiarata esclusa per limite di perimetro della suite unitaria, non trattata come gap di copertura da chiudere: coprirla richiederebbe un test di integrazione tra le due classi, categoria non coperta da questo lavoro.

L'uso dei mock è giustificato in ottica **black-box** in ogni fase: i mock controllano esclusivamente cosa restituiscono i collaboratori, senza richiedere dipendenze reali (database, container Spring), mentre il comportamento verificato resta quello osservabile dall'esterno dei metodi pubblici – lo stesso che vedrebbe un chiamante reale.

Il raffinamento strutturale di Fase 2 segue sistematicamente, per entrambe le classi, la modalità **white-box-dentro-black-box**: la partizione di Fase 1 non viene ridisegnata né ricombinata, si aggiunge un singolo test mirato al ramo scoperto dal report JaCoCo. La scelta resta tracciabile a livello di file: i test di raffinamento risiedono in una classe separata (*WBTest) e adottano una convenzione di naming distinta (RF, Raffinamento Frame) per non lasciare intendere che siano derivati con lo stesso metodo della Category Partition.

Il rafforzamento guidato da PIT segue lo stesso principio di tracciabilità, per un difetto di natura diversa: un mutante sopravvissuto su una riga già coperta indica un'asserzione poco precisa. Il test frame originale non viene mai modificato; la versione rafforzata vive in un terzo file separato (*MTTest), con naming MT.

Il criterio di filtraggio test applicato a Randoop e alle suite LLM è dichiarato in dettaglio nelle sezioni successive del report.

3. Classe: CommandLogic

3.1 Il SUT: responsabilità e collaboratori

CommandLogic è lo strato di logica applicativa che gestisce i *Command*, punti di estensione eseguibili di Syncope registrati come *Implementation* e invocabili con argomenti tipizzati (*CommandArgs*). La classe espone tre metodi pubblici funzionali – search, read, run – più un override tecnico (resolveReference) che si limita a lanciare UnsupportedOperationException e resta quindi fuori da questa analisi. Sono stati selezionati **search(String keyword, Pageable pageable)** e **run(CommandTO command)**: il primo combina due parametri con lo stato del collaboratore *ImplementationDAO* in una logica di paginazione manuale; il

secondo riceve un solo parametro ma di tipo composito ed espone quattro punti di diramazione verso collaboratori esterni (ricerca dell'implementazione, costruzione tramite *ImplementationManager*, validazione *Bean Validation*, esecuzione). *ImplementationDAO* e *Validator* sono mock Mockito standard, iniettati da costruttore; *ImplementationManager* espone solo metodi statici e richiede quindi Mockito.mockStatic.

L'approccio adottato per la Category Partition è **black-box** rispetto alle funzionalità della SUT: le categorie derivano dal dominio semantico dei parametri. Né *CommandLogic* né i suoi collaboratori (*ImplementationDAO*, *Validator*, *ImplementationManager*) dispongono di **Javadoc**: l'inferenza dal codice resta quindi necessaria, ma è dichiarata esplicitamente e limitata a due soli scopi – conoscere i **tipi** e i **contratti** dichiarati dei collaboratori, e, dove nemmeno un contratto dichiarato esiste, verificarne il **comportamento minimo necessario** a configurare mock realistici.

3.2 Category Partition e Boundary Value Analysis

Metodo *search(String keyword, Pageable pageable)*

- **Dimensioni identificate** – dalla firma del metodo più una dimensione ambientale implicita:
 - D1 = keyword;
 - D2 = pageable (unico dominio ordinale della classe);
 - D3 = stato del collaboratore *ImplementationDAO* (non un parametro formale, ma necessario a fornire un input concreto al mock).
- **Categorie, su base semantica (Tabella 1):**
 - keyword: *blank* (nulla o vuota), *valorizzata con corrispondenza*, *valorizzata senza corrispondenza esatta*.
 - *keyword=null* e *keyword=""* non sono trattate come classi distinte, ma come due rappresentanti della stessa classe "blank": l'implementazione JPA del DAO (*ImplementationRepoExtImpl*) applica la **guardia** *StringUtils.isNotBlank(keyword)* prima di aggiungere la clausola LIKE alla query, trattando i due valori in modo identico – nessun filtro applicato, l'intero catalogo restituito. Un secondo test frame con *keyword=""* avrebbe eseguito lo stesso percorso di codice e prodotto lo stesso oracolo di TF2, senza aggiungere potere di rilevazione.
 - pageable: *nominale*, *pageSize=0*, *unpaged*, *null*, *offset uguale alla dimensione del risultato* (limite).
- **Boundary Value Analysis (Tabella 2)** – applicata solo a **pageable**, unica dimensione ordinale: sei rappresentanti di frontiera, tra cui *pageNumber=-1*, valore che viola il vincolo pubblico ≥ 0 e non è raggiungibile tramite le factory standard. Trattato come oracolo da verificare sperimentalmente, non come certezza dedotta dal contratto.
- **Combinazione**: unidimensionale – ogni classe compare in almeno un test frame, evitando il prodotto cartesiano completo ($3 \times 8 \times 3 = 72$ combinazioni, in gran parte ridondanti). Da chiarire che questa scelta non garantisce di esporre interazioni patologiche tra parametri. L'unica combinazione multidimensionale individuata (keyword valorizzata incrociata con un elemento del DAO non convertibile) è stata rinviata alla Fase 2, perché prevederne l'esito richiedeva di leggere il corpo del metodo, quindi riguarda alla conoscenza strutturale.
- **Criterio di filtro**: "una sola anomalia alla volta" – ogni scenario invalido introduce una singola classe anomala, mantenendo gli altri parametri al valore nominale, per non sommare più anomalie nello stesso frame ed evitare ambiguità su quale fattore abbia causato l'esito osservato.
- **Risultato**: 10 test frame (Tabella 3), oracoli dichiarati prima dell'esecuzione (Tabella 4).

Metodo *run(CommandTO command)*

- **Dimensioni identificate** – dalla scomposizione del parametro composito *command* più due esiti di collaborazione impliciti:
 - D1 = *command* (sotto-campi *key*, *args*);
 - D2 = esito di *ImplementationManager.build(...)*;
 - D3 = esito della validazione *Bean Validation*;
 - D4 = esito di *runnable.run(...)*;Nessuna di queste dimensioni è ordinale: a differenza di *search*, qui la BVA non ha rappresentanti naturali (Tabella 5).
- **Categorie, su base semantica**: *command* nullo; *key* nulla / vuota / presente nel DAO / assente nel DAO; *build* riuscito / fallito; *args* nullo / valido / con violazioni; *run* riuscito / con eccezione.
 - La classe "*command.key* vuota" non ha un test frame dedicato: a differenza di *keyword* in *search(...)*, qui non esiste alcuna guardia di *CommandLogic* sulla stringa (nessun controllo di blank-ness prima dell'invocazione del DAO). Una **chiave vuota** è quindi, dal punto di vista del metodo, semplicemente un valore sintatticamente valido che non risolve nel DAO mockato – la stessa classe di equivalenza già rappresentata da TF5 ("missing-cmd"), non un caso di null-safety

(già coperto separatamente da TF4). È stata quindi assorbita nella classe "valida, assente nel DAO" anziché trattata a parte.

- **Combinazione:** unidimensionale, stesso criterio di *search*. Una combinazione prevista in fase preliminare (argomenti con violazioni + validator che solleva un'eccezione direttamente) è stata rimossa dalla Category Partition, perché distinguerne l'esito richiedeva di sapere che entrambi gli scenari convergono sullo stesso blocco catch – anche questa rinviata alla Fase 2.
- **Filtro:** criterio "una sola anomalia alla volta", applicato in modo simmetrico a *search*:
 - *command = null* (TF3) testato da solo, senza incrociarlo con le classi di key/args;
 - key nulla o assente nel DAO (TF4/TF5) testate con un solo valore di args (nullo), senza esplorare tutte le sue classi;
 - *build(...)* che fallisce (TF6) testato con un solo valore di args, per lo stesso motivo.
- **Risultato:** 8 test frame (Tabella 6), oracoli dichiarati prima dell'esecuzione (Tabella 7).

3.3 Esecuzione dei test manuali Black-Box

La suite *CommandLogicBBTest* implementa i 18 test frame di Fase 1 (10 per *search*, 8 per *run*).

L'esecuzione (`mvn test -pl core/idrepo/logic -Dtest=CommandLogicBBTest`) restituisce 18/18 test superati.

Il caso più significativo dal punto di vista metodologico è **TF8** (*pageNumber=-1*, oracolo O7) del metodo *search* (Tabella 3): dichiarato in fase di progettazione come ipotesi da verificare sperimentalmente, poiché derivarlo con sicurezza avrebbe richiesto di leggere il corpo di *search(...)* – conoscenza esclusa dall'approccio black-box di questa fase. L'esecuzione conferma l'ipotesi: il valore produce un *IllegalArgumentException*, coerente con la violazione del vincolo pubblico *pageNumber ≥ 0* dichiarato da *Pageable/PageRequest*. Tutti gli oracoli dichiarati in Fase 1 si confermano coerenti con l'esito osservato, senza necessità di marcare alcun test come *@Disabled*.

La suite è stata successivamente riscritta per adottare *@Mock/@InjectMocks* con *@ExtendWith(MockitoExtension.class)*, al posto dell'attivazione manuale (*MockitoAnnotations.openMocks(this)*), aggiungendo *mockito-junit-jupiter* al pom.xml del modulo.

3.4 Raffinamento strutturale guidato da JaCoCo

Il report JaCoCo (Figura 1) generato dopo l'esecuzione della sola suite black-box (*CommandLogicBBTest*, 18 TF) segna **74% di istruzioni** e **100% di branch** coperti. I branch sono già al 100% con il solo BB: la Category Partition aveva già previsto tutte le decisioni logiche del metodo. Il gap residuo è localizzato: righe 77-79 di *search(...)*, il blocco *catch (Exception e) { return null; }* dentro *.map(...)*, mai esercitato (Figura 2) – coincide esattamente con il caso multidimensionale rinviato dalla Category Partition (Sezione 3.2: keyword valorizzata incrociata con un elemento del DAO non convertibile).

Il gap viene chiuso aggiungendo **RF1** in un file separato (*CommandLogicWBTest.java*), con naming RF. Un secondo report JaCoCo, generato dopo l'esecuzione di *CommandLogicWBTest*, conferma la chiusura: le righe 77-79 passano da non coperte a coperte, e la suite complessiva sale a **77% istruzioni / 100% branch** (+3%, guadagno interamente attribuibile a questa singola lambda) (Figura 3). RF1 rivela inoltre un comportamento non documentato: quando un elemento del DAO fallisce la conversione a *CommandTO*, viene filtrato in silenzio dal risultato (*content*), ma *count* non viene ricalcolato – la pagina restituita riporta un conteggio disallineato dal contenuto effettivo (Esempio 1).

Su *run(...)*, JaCoCo segna già "All 2 branches covered" su ogni punto di decisione con la sola suite BB – nessun gap sui rami logici. Restano scoperte solo le righe 115-116 (Figura 4), le lambda di gestione cache passate a *ImplementationManager.build(...)*: non vengono chiuse con un nuovo test, poiché eseguite solo dal corpo reale di *build(...)*, sempre mockato in questa suite – coerente col limite di perimetro già dichiarato in Sezione 2. *read(...)* e *resolveReference(...)* restano interamente non coperti, a conferma del confine di scope già dichiarato in Sezione 3.1.

Un limite emerge dalla Fase 2 stessa: nessuno dei due casi rinviati dalla Category Partition (il caso multidimensionale di *search*; la convergenza args-con-violazioni / validator-che-lancia-direttamente in *run*) compare come riga rossa o gialla nel report – JaCoCo segna un if come coperto se **qualche** test ha preso entrambi i rami, non se un singolo test ha combinato più condizioni contemporaneamente (limite della Decision Coverage rispetto alla Multiple Condition Coverage). Questi casi restano da valutare in fase di Mutation Testing (Sezione 3.5), non di sola coverage: aggiungendo la suite di rafforzamento (*CommandLogicMTTest*), la copertura resta invariata a 77%/100%, perché il mutation testing rafforza gli oracoli su codice già toccato, non aumenta la coverage strutturale.

3.5 Adeguatezza semantica: Mutation Testing (PIT)

Il **primo run** di PIT (scope su *CommandLogicBBTest* + *CommandLogicWBTest*) genera 11 mutazioni: 7 killed, 3 NO_COVERAGE – coincidenti con i limiti di perimetro già dichiarati (*read()*, *resolveReference()*, le lambda L115-116 di *run()*) – e **1 sopravvissuto** (Figura 5): *NegateConditionalsMutator* su L139, dentro il

ternario che sceglie tra `emptyArgs(...)` e `command.getArgs()`. Mutation score grezzo 64% (7/11); test strength 88%.

La **causa** della sopravvivenza è isolabile con precisione: TF1 e TF2 (in `CommandLogicBBTest`) stubbano `runnable.run(any())` – qualunque `CommandArgs` riceva `run(...)`, lo stub risponde "OK" comunque. Il mutante scambia quale `CommandArgs` viene passato, ma l'asserzione originale controlla solo il valore di ritorno, mai l'argomento: copertura piena della riga, ma asserzione cieca su quello specifico aspetto del comportamento.

Un secondo intervento **white-box**, questa volta guidato da un mutante sopravvissuto. Le **versioni rafforzate** (MT1, MT2: stesso scenario, ma con `eq(argomento_esatto)` al posto di `any()` e `verify(runnable).run(...)` esplicito) vivono in un terzo file separato, `CommandLogicMTTest.java`.

Un **secondo run** di PIT, con `CommandLogicMTTest` incluso nello scope, conferma: L139 passa a KILLED su entrambe le mutazioni presenti su quella riga (**Figura 6**) – incluso un secondo mutante (sostituzione del valore di ritorno con stringa vuota), ucciso "gratuitamente" dallo stesso rafforzamento, senza bisogno di un test dedicato. Test strength: 100% (8/8); mutation score grezzo 73% (8/11, denominatore invariato per via dei 3 NO_COVERAGE già giustificati in Sezione 3.4). Zero mutanti sopravvissuti nell'intero file.

I due casi rinvii dalla Category Partition e non emersi come gap in JaCoCo (Sezione 3.4) – il caso multidimensionale di `search`, la convergenza args-con-violazioni/validator-eccezione in `run` – non emergono nemmeno qui come mutanti sopravvissuti: la suite attuale uccide già implicitamente i mutanti equivalenti a quegli scenari, senza richiedere un test dedicato per nessuno dei due.

3.6 Generazione automatica – Randoop

Dopo la Category Partition manuale e il raffinamento guidato da JaCoCo/PIT, l'obiettivo di questa fase è duplice: esplorare sequenze di input che un tester umano non avrebbe pensato di provare, e verificare se `CommandLogic` nasconda comportamenti anomali non anticipati nella progettazione manuale. La seed factory (`CommandLogicRandoopSeeds`) usa Mockito per fornire a Randoop un modo di costruire un'istanza funzionante – coerentemente col principio dichiarato in Sezione 2, i mock controllano cosa restituiscono i collaboratori, mentre il comportamento esplorato resta quello osservabile dall'esterno dei metodi pubblici. La conoscenza della firma del costruttore, necessaria per scrivere la seed, è dichiarata qui esplicitamente come dettaglio strutturale: questa fase non è vincolata alla purezza black-box di Fase 1.

Esecuzione: Randoop 4.3.4, JDK 25, budget di 60 secondi, classpath costruito via `mvn dependency:build-classpath` (scope test), offline da terminale, genera sequenze di chiamate con un approccio **feedback-directed**. **Risultato:** 199 sequenze di regressione generate, nessun error-revealing test.

La suite grezza (199 sequenze) è curata a **3 test** (RT1-RT3), scartando le sequenze in cui Randoop assegna `null` a un parametro pur avendo a disposizione un producer alternativo (seed factory o classe in classlist) – un pattern statisticamente economico per Randoop ma poco informativo per il tester. Sono mantenute le sequenze in cui gli oggetti passati sono costruiti realmente (`new CommandTO()` con `key/args` valorizzati tramite la seed factory), perché solo queste attraversano i punti di diramazione verso `ImplementationDAO/ImplementationManager/Validator` invece di terminare subito su `NullPointerException`. La suite curata copre 25% delle istruzioni e 0% dei branch (**Figura 7**): un valore basso ma atteso, perché anche le sequenze sopravvissute alla curation raramente attraversano più di un ramo decisionale ciascuna.

RT1/RT2 (`run(...)`) osservano che, con una chiave assente o esplicitamente vuota, il messaggio dell'eccezione `NotFoundException` incorpora la chiave passata ("Implementation null" vs "Implementation ") – comportamento parametrico, non fisso, scoperto da Randoop esplorando valori di stringa diversi. Classificato come comportamento corretto, non un bug: il metodo gestisce esplicitamente il caso "implementazione non trovata" con un'eccezione di dominio informativa. **RT3** (`search(...)`) conferma che con `keyword/pageable` reali il metodo restituisce sempre una `Page<CommandTO>` non nulla, mai un'eccezione – con il DAO mockato non stubbato la pagina risulta vuota (comportamento di default documentato di Mockito sulle collection), poco informativo sul contenuto ma di conferma sulla robustezza del metodo rispetto a input arbitrari.

3.7 Generazione automatica – LLM (Zero-shot, Few-shot, Guided ToT)

Le tre tecniche condividono **strumento** (Microsoft 365 Copilot, modalità "Think Deeper"), e **perimetro** (`search()/run()`, coerente con CP e Randoop) per garantire un confronto controllato: cambia solo il contenuto del prompt (**Prompt 1**, **Prompt 2**, **Prompt 3**). Sulla coverage per tecnica (**Tabella 8**, **Tabella 9**) e complessivo (**Figura 8**), le tre saturano lo stesso tetto (77%/100%). Il **criterio di validazione** è lo stesso per tutte e tre: eseguire il file grezzo così com'è, passare l'errore a Copilot nello stesso thread per un tentativo di auto-correzione, intervenire manualmente solo come ultima istanza. Ogni output grezzo integrale è conservato in docs/llm-raw/ del progetto.

Zero-shot (Prompt 1) – 9 test generati.

- **Validazione statica** (lettura critica, prima di eseguire): 2 errori di ragionamento individuati nel codice generato:
 - *searchReturnsRequestedPageWithCommandTransferObjects* (**Listato 2**) – errore di **logica di paginazione**: il test si aspetta 2 elementi da *PageRequest.of(1,2)* su un DAO con 3 elementi totali, coerente con un offset calcolato come *pageNumber* invece di *pageNumber × pageSize*; con l'offset corretto (2) la pagina contiene un solo elemento.
 - *runThrowsInvalidEntityExceptionWhenValidationViolationsExist* – asserzione sul tipo di eccezione sbagliata: non tiene conto che *CommandLogic* intercetta *InvalidEntityException* e la ri-lancia come *SyncopeClientException*.
- **Compilazione**: il file grezzo non compila (7 errori con stessa causa) – *when(implementationDAO.findById(...)).thenReturn(...)* incompatibile col bound generico wildcard della gerarchia DAO, non visibile all'LLM perché non inclusa nel source fornito (**Figura 9**). Corretto manualmente con l'idioma *doReturn(...).when(...)*, lo stesso già usato in tutta la suite manuale.
- **Primo run**: conferma empiricamente entrambi gli errori già previsti in validazione statica.
- **Correzione affidata a Copilot**: sul test di paginazione, ripara cambiando lo scenario da *PageRequest.of(1,2)* a *(0,2)* – evita il caso con offset>0 invece di correggerne il calcolo: risultato corretto, ma per aggiramento. Sul test dell'eccezione, non ripara: stubba *getMessage()* invece di *getMessageTemplate()* (l'unico metodo realmente letto dal costruttore di *InvalidEntityException*) e non stubba *getLeafBean()*.
- **Fix manuale finale**: lettura diretta del costruttore di *InvalidEntityException*, 4 stub corretti.
- **Esito**: 9/9 test – 6 sovrapposti a test BB/MT esistenti, 2 scenari nuovi (fallimento parziale di *emptyArgs()* in *search()*; propagazione di un'eccezione diretta del Validator).

Few-shot (Prompt 2) – 12 test generati, con 2 esempi forniti (TF1 e TF7 dalla suite BB manuale, solo i metodi @Test, senza gli helper privati).

- **Validazione statica**: l'idioma *doReturn(...).when(...)* viene usato correttamente ovunque fin dalla generazione – appreso dall'esempio TF7, che già lo adottava. 2 test quasi identici agli esempi forniti (tf1_nominalSearch, tf7_argsWithViolations): bassa evidenza indipendente della tecnica, effetto noto del few-shot prompting. Previsto, per lettura critica, un bug analogo a quello già trovato nello zero-shot: l'helper locale *mockViolation()* con stub incompleto rispetto al costruttore reale di *InvalidEntityException*.
- **Compilazione**: zero errori.
- **Primo run**: 8 problemi su 3 cause distinte:
 - 4 *UnnecessaryStubbingException* (tf2, tf4, tf6, tf11) – l'helper *mockImplementation()*, ricostruito da Copilot, non usa *lenient()*: il prompt few-shot conteneva solo i metodi @Test degli esempi, non l'implementazione dell'helper.
 - 3 *AssertionFailedError* su *assertEquals(List.of(...), sce.getElements())* (tf9, tf10, tf12) **Figura 10** – *getElements()* restituisce *Set<String>*, non *List*: *List.equals()* e *Set.equals()* non sono mai true tra loro per contratto del Java Collections Framework, indipendentemente dal contenuto.
 - 1 *NullPointerException* (tf7) – stesso stack trace dello zero-shot: *mockViolation()* stubba solo *getMessage()*, mai *getMessageTemplate()*.
- **Correzione affidata a Copilot**: ripara correttamente sia l'helper (*lenient()* aggiunto, **Figura 11**) sia l'asserzione (*Set.of(...)* al posto di *List.of(...)* – cambiato il tipo atteso invece del metodo di confronto, ma funzionalmente corretto). Fallisce per la **terza volta consecutiva** sullo stesso punto: aggiunge *getPropertyPath()* e *getInvalidValue()*, ma continua a stubbare *getMessage()* invece di *getMessageTemplate()*, e non stubba *getLeafBean()*.
- **Fix manuale finale**: identico a quello già verificato sullo zero-shot (4 stub reali).
- **Esito**: 12/12 integrati – 2 quasi identici agli esempi forniti, 2/12 nuovi rispetto al manuale, 1/12 nuovo anche rispetto allo zero-shot LLM.

Guided ToT (Prompt 3) – 13 test generati, fornendo cinque step guidati con tre esperti simulati.

- **Validazione statica**: 2 scenari nuovi, mai comparsi in nessuna tecnica precedente né nel manuale – robustezza su **pageable nullo** (*searchShouldThrowNullPointerExceptionWhenPageableIsNull*) e propagazione di **un'eccezione del DAO** mai catturata da *search()* (*searchShouldPropagateDaoException*). Le asserzioni sulle eccezioni sono in generale più deboli di zero-shot/few-shot (*assertNotNull(exception)* invece di verificarne tipo/contenuto).
- **Compilazione**: **BUILD FAILURE, 7 errori** – stessa identica causa dello zero-shot (bound generico wildcard sul DAO).
- **Correzione affidata a Copilot** (solo log di compilazione): risolve i 7 errori, ma **introduce un bug nuovo**, assente nell'originale, in *runShouldThrowSyncopeClientExceptionWhenImplementationCannotBeBuilt*, la chiave stubbata viene

cambiata per errore da "brokenCommand" (corretta) a "command1" (copiata da un altro test), producendo a runtime *PotentialStubbingProblem* invece del comportamento atteso..

- **Primo run eseguibile:** 8 problemi: 6 *UnnecessaryStubbingException* (helper *implementation()* senza *lenient()*), 1 *PotentialStubbingProblem* (il bug appena introdotto dalla correzione), 1 *NullPointerException* (bug su *ConstraintViolation/InvalidEntityException*, **Esempio 2**).
- **Fix manuale finale:** *lenient()* aggiunto; chiave corretta da "command1" a "brokenCommand"; violation ristubbato con i 4 getter reali, asserzione corretta da *assertThrows(InvalidEntityException.class, ...)* a *assertEquals(ClientExceptionType.InvalidValues, exception.getType())*.
- **Esito:** 13/13 integrati, BUILD SUCCESS – 2 scenari nuovi rispetto a tutte le tecniche precedenti e al manuale.

3.8 EvoSuite – N/A

Il tentativo di generazione automatica strutturale (EvoSuite) è N/A per *CommandLogic*, per un vincolo più forte della semplice incompatibilità di versione: non esiste, per costruzione, alcuna versione storica della classe compilata in un bytecode che EvoSuite possa accettare (major version ≤52, limite noto del tool, ultima release 2021). *CommandLogic* è stata introdotta ex novo nella release 3.0.0 di Syncope, il cui pom dichiara già targetJdk 11 (bytecode major version 55), verificato su tutti i tag di release principali, sia sul path corrente sia su quello storico pre-restructuring: la classe è assente su ogni release precedente. Il progetto corrente la compila in major version 69 (JDK 25).

3.9 Evoluzione delle varianti C1–C4

Le quattro varianti applicano lo stesso smell SonarCloud (e inutilizzato in un catch) con ampiezza crescente, misurabile sui catch toccati:

Variante	Catch corretti (su 5)	Modifiche fuori scope	Compila
C1 (= C4)	2 (solo dove e è inutilizzato)	Nessuna	Sì
C2	5 – anche dove e era usato: rimosso da LOG.error() e da sce.getElements().add(...), perdendo stack trace e messaggio d'eccezione	2 nuovi smell "clumsy"	Sì
C3	5, stessa strategia di C2	resolveReference() riscritto, raw type, ClientExceptionType.RunError mai verificato, domain rimosso solo in 2/3 metodi	No

La generazione automatica sulle varianti segue esattamente lo stesso approccio già usato su C0, stesso strumento, stesso perimetro (*search()/run()*), stesso ciclo di validazione (eseguire il grezzo, correggere con Copilot, intervenire manualmente come ultima istanza): cambia solo il sorgente allegato al prompt.

Per ogni variante comportamentalmente distinta si genera ex novo test Randoop e LLM (Zero-shot, ToT).

C1: 31/31 test, nessuna regressione. **C2 (finding centrale): 1 sola asserzione, di 1 sola tecnica** (Zero-shot), rileva la regressione reale sul messaggio d'eccezione (**Esempio 3**), le altre tecniche si fermano al tipo di eccezione, non al contenuto. Il test viene marcato @Disabled con motivazione, come richiesto per ogni test il cui fallimento è comportamento atteso: PIT rifiuta di eseguire l'analisi di mutazione su una suite non verde, quindi disabilitarlo è l'unico modo per misurare comunque l'adeguatezza su questa variante, la coverage riportata in **Tabella 11** (75%) resta quella a test attivo, la reale copertura della suite consegnata. **C3:** non compila (6 errori). **C4:** omesso, byte-identico a C1, risultati C1 riusati.

Un secondo controllo, complementare alla generazione ex-novo appena descritta, riesegue le suite già esistenti su C0 (manuale, Randoop, LLM) direttamente sulle varianti compilate, senza rigenerare nulla (**Tabella 10**): su C1 e C4 il risultato è pieno (58/58); su C2, **4 dei 34 test LLM originali falliscono** – due Few-shot e due Zero-shot, tutti per la stessa causa: l'asserzione su *sce.getElements()* si aspetta il messaggio originale dell'eccezione, che C2 non aggiunge più al *catch*. Né la suite manuale (21/21) né Randoop (3/3) rilevano la regressione, e nemmeno Guided ToT (13/13).

Confronto Randoop/Zero-shot/ToT su C0 vs varianti (Tabella 11, Tabella 12): Randoop resta strutturalmente piatto su tutte e tre le varianti – 0% branch e 27% (3/11) di mutation score identici su C0, C1, C2, con solo una leggera oscillazione nelle istruzioni (25%→25,1%→28%) dovuta al numero diverso di righe totali per variante, non a una suite migliore o peggiore: è la stessa conferma quantitativa, ora estesa a tutte e tre le varianti, del limite strutturale già discusso in Sezione 3.6 (Randoop non arriva mai al codice che il refactoring modifica). Le **due tecniche LLM** restano invece stabili su un tetto molto più alto (coverage 75-77%, mutation 73%, test strength 100% su ogni variante), con le stesse oscillazioni minori già descritte (ToT tiene il 100% di branch anche su C1, Zero-shot scende a 66,7% su C1; entrambe scendono a 75% di istruzioni su C2 per il denominatore più piccolo, non per una suite peggiore). Il divario Randoop/LLM

osservato su C0 non si restringe né si allarga sulle varianti: il refactoring cambia il comportamento della classe, non la capacità delle due tecniche di esplorarlo.

3.10 Stima di reliability

Metodologia (vale anche per Sezione 4.10): $Reliability = 1 - PFD$, con $PFD = \text{fallimenti osservati} / N$, assumendo un profilo operativo a **probabilità uniforme** sull'insieme finale di test. Per "fallimento" si intende un difetto realmente presente nella classe, non un errore nel test stesso: questi ultimi sono già stati individuati e corretti nel ciclo di validazione critica documentato per ciascuna suite.

CommandLogic: $N = 59$ (BB 19, WB 1, MT 2, Randoop 3, Zero-shot 9, Few-shot 12, ToT 13), nessuna esclusione. Fallimenti osservati: **0**. $PFD = 0 \rightarrow Reliability = 1.0$.

Un valore pari a 1.0 non indica una suite perfetta o una classe priva di difetti in assoluto: indica che, sui 59 scenari effettivamente esplorati – un campione rappresentativo ma non esaustivo del dominio di input – non è emerso alcun fallimento. Scenari fuori da questo campione restano per costruzione non verificati.

4. Classe: NotificationDataBinderImpl

4.1 Il SUT: responsabilità e collaboratori

NotificationDataBinderImpl (core/provisioning-java/data) implementa l'interfaccia *NotificationDataBinder* (anch'essa priva di Javadoc, verificato sui sorgenti) ed espone tre metodi pubblici: *getNotificationTO(Notification)*, *create(NotificationTO)*, *update(Notification, NotificationTO)*. Sono stati selezionati **update(...)** e **getNotificationTO(...)**, le due direzioni complementari della classe: *update(...)* è il metodo più ricco, dove *NotificationTO* ha oltre dieci campi, diversi dei quali associati a un comportamento osservabile distinto (eccezioni di tipo diverso, filtri silenziosi, campi ignorati se non risolvibili), ed è l'unico a interagire con tutti e cinque i collaboratori della classe; *getNotificationTO(...)* decompone lo stesso oggetto complesso nella direzione opposta (entità $\rightarrow$ TO), con una struttura più semplice ma un ramo dichiarato distinto (gestione di *recipientsProvider* nullo).

update(...) interagisce con **cinque collaboratori**, tutti iniettati da costruttore (nessuna dipendenza da metodi statici, quindi la loro simulazione richiede solo mock Mockito standard): *mailTemplateDAO* risolve la chiave del template in un'entità reale, determinando se il campo è "trovato" o mancante; *anyTypeDAO* valida che i tipi di AnyObject referenziati nei destinatari esistano nel sistema; *implementationDAO* risolve l'implementazione del *recipientsProvider*, quando dichiarato; *intAttrNameParser* valida la sintassi degli attributi interni referenziati nei filtri, indipendentemente dalla loro esistenza. *entityFactory* crea una nuova istanza di *AnyAbout* quando *update(...)* deve aggiungere un *about* non ancora presente nell'entità, distinguendo questo caso da quello in cui un *about* esistente va aggiornato in-place invece che ricreato. *notification* (in entrambi i metodi) è un'interfaccia e va mockata per rappresentare i suoi stati rilevanti.

Vale lo stesso vincolo di approccio **black-box** dichiarato in Sezione 3.1: né la classe né le sue dipendenze hanno Javadoc, quindi l'inferenza dal codice resta necessaria per conoscere i **tipi** coinvolti e il **contratto** osservabile.

4.2 Category Partition e Boundary Value Analysis

Metodo *update(Notification notification, NotificationTO notificationTO)*

- **Dimensioni identificate** (Tabella 13): un solo parametro esplicito ma composito (*notification*, D1) più sei sotto-campi di *notificationTO*, selezionati perché ciascuno è associato a un comportamento osservabile distinto – un'eccezione specifica o un filtro dichiarato (D2.1-D2.6: *template*, *events*, *staticRecipients*, *abouts*, *recipientAttrName*, *recipientsProvider*) – e una dimensione ambientale aggiuntiva (D3, lo stato iniziale di *notification.getAbouts()*), necessaria perché *update(...)* non sovrascrive gli *about* esistenti ma li riconcilia con quelli nuovi: senza rappresentare cosa c'era già nell'entità, non è possibile costruire uno scenario che osservi una rimozione. I restanti campi di *notificationTO* (*key*, *recipientsFIQL*, *selfAsRecipient*, *sender*, *subject*, *traceLevel*, *active*) sono copiati senza alcun ramo osservabile associato: trattati come un'unica classe, nessun test frame dedicato.
- **Categorie, su base semantica**: 8 categorie in totale – tra le più rilevanti, *template* (con corrispondenza in *mailTemplateDAO* / senza corrispondenza / nullo), *staticRecipients* (vuota / indirizzi conformi a Entity.EMAIL_PATTERN / almeno uno non conforme), *abouts* (vuota / chiavi tutte risolvibili con valori non-blank / almeno una chiave non risolvibile / almeno un valore blank).
- **Boundary Value Analysis**: non applicabile, dato che nessuna delle dimensioni è ordinale.
- **Combinazione**: unidimensionale, dato che il prodotto cartesiano completo genererebbe $2 \times 3 \times 2 \times 3 \times 4 \times 3 \times 3 \times 2 = 2.592$ combinazioni.
- **Filtro**: criterio "una sola anomalia alla volta" – ogni scenario invalido introduce una singola classe anomala, resto nominale.
- **Risultato**: 11 test frame (Tabella 14), oracoli definiti prima dell'esecuzione (Tabella 15). Due ipotesi dichiarate esplicitamente come da verificare sperimentalmente, non come certezze: *notification = null*

produce eccezione (comportamento non documentato); un *about* non più presente in *notificationTO* viene rimosso dall'entità, coerente con una semantica di sostituzione completa tipica di un'operazione "update".

Metodo *getNotificationTO(Notification notification)*

- **Dimensioni identificate (Tabella 16):** un solo parametro esplicito ma composito (*notification*), scomposto in cinque sotto-campi (D1.1-D1.5: *template*, *events*, *staticRecipients*, *abouts*, *recipientsProvider*) – gli stessi campi trattati come sotto-domini in *update(...)*, qui riletti nella direzione opposta (entità → TO). Gli altri campi semplici (*key*, *recipientsFIQL*, *recipientAttrName*, *selfAsRecipient*, *sender*, *subject*, *traceLevel*, *active*) passano da *notification* a *notificationTO* con una copia diretta getter→setter, senza alcun controllo o ramo condizionale: nessun comportamento osservabile da isolare, quindi nessun test frame dedicato
- **Categorie:** per i sotto-campi composti (*template*, *abouts*), essendo tipi a riferimento non primitivi, il valore nullo è sintatticamente ammesso dal tipo del getter indipendentemente da un contratto dichiarato: la classe "nullo" è quindi inclusa come partizione a sé, non un'aggiunta arbitraria.
- **Boundary Value Analysis:** non applicabile.
- **Combinazione:** unidimensionale, dato che $2 \times 3 \times 3 \times 3 \times 2 = 108$ combinazioni nel prodotto cartesiano, in gran parte ridondanti.
- **Filtro:** stesso criterio "una sola anomalia alla volta".
- **Risultato:** 9 test frame (Tabella 17) oracoli in Appendice (Tabella 18). Un'unica ipotesi trasversale: ogni sotto-campo composto nullo produce eccezione, copiato senza controllo visibile – da verificare su ciascuno dei quattro sotto-campi interessati.

4.3 Esecuzione dei test manuali Black-Box

La suite *NotificationDataBinderImplBBTest* implementa i 20 test frame di Fase 1 (11 per *update*, 9 per *getNotificationTO*). L'esecuzione (`mvn test -pl core/provisioning-java -Dtest=NotificationDataBinderImplBBTest`) restituisce 20/20 test superati, incluse tutte e tre le ipotesi dichiarate come da verificare sperimentalmente in Sezione 4.2.

Lo scenario più significativo è **TF11 (Figura 12)**: TF11 verifica un'ipotesi sul **comportamento di dominio** di *update(...)*, non deducibile dalla sola firma. Il metodo riceve *notification* (l'entità esistente, con i suoi *abouts* correnti) e *notificationTO* (i valori nuovi): non è ovvio, senza leggere il codice, se *update* debba **aggiungere** i nuovi *abouts* lasciando invariati quelli esistenti (semantica additiva) o **sostituire** interamente l'insieme (semantica di reset completo) – due contratti ugualmente plausibili per un metodo chiamato "update", con conseguenze opposte per chi lo invoca. Lo scenario costruisce un *about* già presente nell'entità per un *AnyType* ("OTHER_TYPE") assente dalla *notificationTO* in ingresso (vuota); l'esecuzione conferma la semantica di sostituzione completa: l'*about* esistente viene rimosso, non conservato. È l'unico test della suite BB il cui esito non era deducibile a priori dalla firma del metodo.

Le restanti ipotesi confermate – *notification* = null (TF2, O2) e i quattro sotto-campi composti nulli (*template*, *events*, *staticRecipients*, *AnyType* annidato in *abouts*) in *getNotificationTO* (TF2-TF5, O2) – restano verificabili direttamente dalla legenda oracoli in Appendice, sono *NullPointerException* dirette.

4.4 Raffinamento strutturale guidato da JaCoCo

Il report JaCoCo dopo la sola suite black-box (*NotificationDataBinderImplBBTest*, 20 TF) segna 96% di istruzioni e 88% di branch, **Figura 13**. A differenza di *CommandLogic*, qui il gap sulle istruzioni non è un raffinamento da colmare: *create(NotificationTO)* resta interamente scoperto per scelta di scope dichiarata fin da Sezione 4.1.

Il gap sul branch (88%) è concentrato su due punti collegati – L151 (*if (about == null)*, ramo "about già esistente" mai esercitato) e L164 (*removeIf(...)*, ramo "about da mantenere" mai esercitato) – coerenti col fatto che la Category Partition di Fase 1 aveva coperto solo il caso complementare: TF11 (Sezione 4.3) verifica che un *about* assente dalla TO venga rimosso, ma nessun test frame verificava il caso opposto, un *about presente* sia nell'entità sia nella TO, che deve essere aggiornato in-place e mantenuto.

Il gap è chiuso da **RF1**, un unico scenario in file separato (*NotificationDataBinderImplWBTest.java*), che copre entrambi i branch contemporaneamente: uno stato iniziale con un *AnyAbout* il cui *AnyType* è presente anche nella *notificationTO* in ingresso – l'esatto opposto di TF11. Un secondo report JaCoCo (**Figura 14**) conferma la chiusura: 96% istruzioni (invariato, *create()* resta fuori scope) / 100% branch (entrambi i rami chiusi dallo stesso scenario). L'**oracolo** di RF1 conferma il comportamento atteso: l'*about* esistente viene aggiornato in-place (nessuna nuova istanza da *entityFactory*, nessuna chiamata a *notification.add(...)*) e resta nella lista dopo la riconciliazione – simmetrico e complementare a TF11.

4.5 Adeguatazza semantica: Mutation Testing (PIT)

Il **primo run** di PIT (scope su BB+WB), **Figura 15**, segna line coverage 96% ma test strength solo 55% (23/42 mutanti raggiunti uccisi): un divario molto più ampio di quello osservato su *CommandLogic* (dove con scope su BB+WB arrivava all'88% di test strength). La causa è sistematica: le mutazioni sui controlli condizionali (validità di template, eventi, indirizzi email, parsing dell'attributo destinatario) muoiono quasi tutte già col solo BB, coerente con quanto la Category Partition è progettata per intercettare; le mutazioni sulle operazioni di copia diretta di campo (*setKey*, *setRecipientsFIQL*, *setRecipientAttrName*, *setSelfAsRecipient*, *setSender*, *setSubject*, *setTraceLevel*, *setActive* – ripetute sia in *getNotificationTO()* sia in *update()*) sopravvivono quasi tutte: le asserzioni della Category Partition controllano un sottoinsieme rappresentativo di campi, non l'intero oggetto risultante – sufficiente a verificare un output ragionevole, non a rilevare la sparizione di un singolo campo dal mapping.

19 mutanti sopravvissuti al primo run, tutti *VoidMethodCallMutator* (rimozione di una chiamata a un setter void), riconducibili a tre pattern: **MT1** (8 mutanti, *getNotificationTO*) rafforza l'asserzione sui campi generici mai controllati in Fase 1; **MT2** (9 mutanti, *update*) verifica con *verify(...)* gli stessi campi generici sul lato opposto, più due setter (*setAnyType*, *setNotification*) sul ramo "crea nuovo about"; **MT3** (2 mutanti, *update*) rafforza *.clear()* su *staticRecipients/events* – invisibile in Fase 1 perché ogni scenario BB parte da liste già vuote, dove rimuovere *.clear()* non cambia nulla di osservabile: serve uno scenario con contenuto preesistente per rilevarlo (**Figura 16**).

Un **secondo run** di PIT (**Figura 17**), con *NotificationDataBinderImplMTTest* incluso nello scope insieme a BB e WB, misura l'effetto dei nuovi test introdotti per uccidere i 19 mutanti sopravvissuti (MT1, MT2, MT3): il **mutation score** sale da 52% (23/44, BB+WB) a **95%** (42/44, BB+WB+MT) – un incremento di **43 punti**, contro i soli 9 punti dello stesso passaggio su *CommandLogic*. La **test strength**, sui soli mutanti effettivamente raggiunti, passa in parallelo da 55% a 100%. La differenza rispetto a *CommandLogic* è spiegabile dalla diversa composizione interna delle due classi: *CommandLogic* ha poca logica di puro mapping di campo da rinforzare, mentre *NotificationDataBinderImpl* è quasi interamente costituita da operazioni di questo tipo (*setter* ripetuti su più campi, sia in lettura sia in scrittura) – da cui il numero elevato di mutanti dello stesso pattern (*VoidMethodCallMutator* su *setter* mai verificati da un'asserzione) e l'effetto sproporzionato dei tre test di rinforzo su *NotificationDataBinderImpl*. I 2 mutanti non raggiunti da nessuna delle tre suite coincidono col gap di copertura già dichiarato su *create()* (Sezione 4.4).

4.6 Generazione automatica – Randoop

Dopo la Category Partition manuale e il raffinamento guidato da JaCoCo/PIT, esploriamo sequenze di input con Randoop. La seed factory (*NotificationDataBinderImplRandoopSeeds*) usa Mockito per fornire a Randoop un modo di costruire un'istanza funzionante – richiede 9 collaboratori mockati in totale: i 5 iniettati direttamente nel costruttore più 4 necessari a costruire *IntAttrNameParser* secondo il suo costruttore reale (*PlainSchemaDAO*, *DerSchemaDAO*, *AnyUtilsFactory*, *RealmUtils*), scoperto verificando il sorgente dopo che un'ipotesi iniziale di costruttore vuoto si è rivelata sbagliata. Coerentemente col principio dichiarato in Sezione 3.6, i mock controllano cosa restituiscono i collaboratori, mentre il comportamento esplorato resta quello osservabile dall'esterno dei metodi pubblici (*update*, *getNotificationTO*).

Esecuzione: stesso setup di Sezione 3.6 (Randoop 4.3.4, JDK 25, 60 secondi di budget, offline, non in CI). **Risultato:** 209 sequenze di regressione più **1 error-revealing test** – una categoria distinta, isolata dal contract di default di Randoop indipendentemente dalla curation delle sequenze di regressione, perché nessun contratto dice se un'eccezione osservata sia lecita.

Il **criterio di filtraggio** applicato alle 209 sequenze di regressione segue tre passaggi, nell'ordine in cui vengono verificati su ogni sequenza: prima si scarta chi esercita solo classi di supporto (getter/setter di *NotificationTO*) senza mai invocare *update()/getNotificationTO()*; poi si scarta chi assegna *null* a *notification/notificationTO* pur avendo un producer disponibile in classlist; infine, tra i superstiti, si tiene un solo rappresentante per ogni causa profonda condivisa, mantenendo più rappresentanti solo se la causa verificata differisce anche a fronte dello stesso messaggio d'eccezione. La suite converge su **3 test** (RT1-RT3, **Tabella 19**): RT1 e RT2 restano entrambe nonostante l'esito identico, perché la causa verificata sul sorgente è diversa – accumulo di campi mancanti per RT1, fallimento di lookup via *mailTemplateDAO* per RT2 – esattamente il caso che il terzo criterio esiste per distinguere.

RT3 (*getNotificationTO()* con *Notification* mock "nudo") è il finding più interessante: *NullPointerException* a *notification.getTemplate().getKey()* (**Esempio 4**), verificato non riproducibile su dati reali – l'entità JPA dichiara il campo *template* con vincolo *optional = false* a livello di schema, quindi irraggiungibile con un *Notification* realmente persistito. Non un bug funzionale: mantenuto in suite come documentazione dell'invariante implicita, escluso dal calcolo di reliability.

Sulla forza dell'oracolo: la suite curata copre 32% delle istruzioni / 22% dei branch (JaCoCo) (**Figura 18**), con test strength PIT al 20% (3/15) (**Figura 19**). La causa è isolabile: *update(Notification, NotificationTO)* è un metodo void. L'oracolo di regressione di Randoop cattura tipicamente il valore di ritorno osservato; quando il metodo non restituisce nulla e la sequenza non ispeziona esplicitamente lo stato del mock mutato, alcune

sequenze raggiungono comunque i setter interni senza però essere in grado di rilevarne l'assenza. La forza dell'oracolo di regressione automatico dipende dalla forma dell'interfaccia del metodo generato (valore di ritorno osservabile o solo effetto collaterale su un mock).

4.7 Generazione automatica – LLM (Zero-shot, Few-shot, Guided ToT)

Stesso setup dichiarato in Sezione 3.7 (Microsoft 365 Copilot, "Think Deeper"), stesso perimetro (*update()/getNotificationTO()*, coerente con CP e Randoop) e stesso criterio di validazione (eseguire il grezzo, correggere con Copilot, intervenire manualmente come ultima istanza). A differenza di *CommandLogic*, qui la Category Partition manuale è già molto esaustiva: nessuna delle tre tecniche produce classi di equivalenza nuove rispetto ai 20 TF esistenti.

Zero-shot (Prompt 4) – 10 test generati.

- **Validazione statica:** emerge una differenza qualitativa di rilievo. La suite BB verifica *update()* prevalentemente per **interazione** (*verify(notification).setXxx(...)*); questo batch introduce invece un helper con *doAnswer* che ricostruisce lo stato mutato e lo verifica per **valore** (*assertEquals* sui campi finali), uno stile diverso, in grado di intercettare classi di bug diverse. Un bug è previsto per lettura critica: il fraintendimento della semantica "chiave presente con valore blank" contro "chiave assente" nella rimozione degli *about*.
- **Compilazione:** 19 errori, due cause (**Figura 20**): Causa A – wildcard sui DAO; Causa B – *Notification.getEvents()/getAbouts()* dichiarati *Set* invece di *List* nel file grezzo, 9 occorrenze (perché l'interfaccia *Notification* non era nel prompt).
- **Correzione affidata a Copilot:** entrambe le cause risolte correttamente al primo tentativo.
- **Primo run eseguibile:** conferma l'unico fallimento già previsto in validazione statica.
- **Correzione affidata a Copilot** (incollando solo log del fallimento runtime): riparato al primo tentativo, asserzioni identiche a quelle già ipotizzate a mano; il metodo viene anche rinominato in modo più coerente col comportamento reale.
- **Esito:** 10/10, BUILD SUCCESS.

Few-shot (Prompt 5) – 13 test, 2 esempi (TF1 e TF2).

- **Validazione statica:** nessuno scenario nuovo rispetto a BB o allo zero-shot LLM. 1 test isola bene un ramo non testato singolarmente da nessun TF manuale (*about* esistente aggiornato vs. nuovo *about* creato).
- **Compilazione:** 2 errori (**Figura 21**): un tipo a wildcard mal gestito e un *when(...).thenReturn(...)* al posto di *doReturn(...).when(...)*.
- **Causa isolata:** gli esempi few-shot includevano solo i metodi *@Test*, non l'helper privato che ricostruisce lo stato mutabile di *abouts*: l'esempio mostra il comportamento atteso di un singolo metodo, non l'infrastruttura di supporto che lo rende eseguibile.
- **Correzione affidata a Copilot** (log di compilazione): entrambi risolti: riscrittura completa dell'helper, tutto tipizzato *List<AnyAbout>* concreto, ma **introdotto 1 nuovo errore**: *verify(intAttrNameParser).parse(...)* (dichiara *throws ParseException*) chiamato come istruzione diretta senza *throws* in firma. **Causa sottile:** *assertDoesNotThrow(...)* scherma le eccezioni checked dentro la lambda che gli viene passata, ma una *verify(...)* scritta fuori da quella lambda no.
- **Fix manuale:** banale, *throws ParseException* aggiunto alla firma.
- **Primo run eseguibile:** Tests run: 13, Errors: 3 – 3 *UnnecessaryStubbingException* sull'helper *mockNotification()*: gli stub su *getAbouts()/getRecipientAttrName()* non sono sempre raggiunti, perché 3 scenari escono da *update()* prima, al controllo *if (!sce.isEmpty()) throw sce*. Lo stesso principio è già documentato esplicitamente nella suite BB stessa, che Copilot non aveva applicato ricostruendo l'helper da zero.
- **Correzione affidata a Copilot** (solo log runtime): riparato al primo tentativo – *lenient()* aggiunto esattamente sui 2 stub corretti, identico all'approccio già nella BB.
- **Esito:** 13/13, BUILD SUCCESS.

Guided ToT (Prompt 6) – 11 test, nessun esempio.

- **Validazione statica:** su 11 test, 9 chiamano *create(...)* invece di *update(...)* – sostituzione silenziosa del metodo sotto test, non un errore che impedisce l'esecuzione, scoperta solo in validazione complessiva, non riga per riga durante la correzione degli errori di compilazione. Non invalida i test (*create()* è un wrapper sottile su *update()*, la stessa validazione viene esercitata indirettamente, e aggiunge un'interazione mai verificata altrove, *verify(entityFactory).newEntity(Notification.class)*), ma va dichiarata esplicitamente come deviazione dal perimetro concordato.

- **Compilazione:** 20 errori, tre cause (**Figura 22**): Causa A – wildcard sui DAO, Causa B – Set/List (quest'ultima riemessa perché nessun esempio insegnava l'idioma corretto, a differenza del few-shot), Causa C – su *IntAttrNameParser/IntAttrName*.
- **Correzione affidata a Copilot** (incollando log di compilazione, primo giro): Causa A risolta su 9/10 occorrenze con un pattern alternativo (*.thenAnswer(invocation -> Optional.of(x))*), ugualmente valido, bypassa la capture conversion per una via diversa da *doReturn*; Causa B e C risolte. Restano 3 occorrenze isolate di Causa A non convertite. BUILD FAILURE, 3 errori residui.
- **Correzione affidata a Copilot** (log di compilazione, secondo giro): le 3 occorrenze residue risolte con lo stesso pattern *.thenAnswer(...)*.
- **Primo run eseguibile:** Tests run: 11, Errors: 3 – *UnnecessaryStubbingException* sullo stesso stub di *getAbouts()/getRecipientAttrName()*.
- **Correzione affidata a Copilot** (solo log runtime): riparata al primo tentativo, *lenient()* aggiunto correttamente sui 2 stub interessati.
- **Esito:** 11/11 integrati, BUILD SUCCESS.

Osservazione trasversale: ogni bug incontrato (logico, di compilazione, o di stub) è stato risolto da Copilot al primo tentativo, senza bisogno di intervento manuale sulla causa. Sulla coverage (**Tabella 20**): Zero-shot eguaglia il tetto manuale; Few-shot e ToT perdono branch sulla gestione degli *about*, per la stessa causa tecnica: l'helper di supporto non simula realmente l'effetto di *notification.add(...)* sul mock. Con ToT più colpito per la deviazione di scope, che sposta il carico dei test dai branch di *update()* a quelli di *create()*.

4.8 EvoSuite – N/A

A differenza di *CommandLogic*, qui esiste una versione storica compatibile: *syncope-core-provisioning-java:2.1.0* (Maven Central), bytecode major version 52 confermato via javap. Il tentativo di generazione su questo artifact si blocca però in modo riproducibile durante la comunicazione interna master-client del tool, senza completare né produrre output, tre correttivi mirati e indipendenti (moduli JPMS, esecuzione client su thread, stack IPv4) non risolvono il blocco.

Per isolare se la causa fosse ambientale o legata alla classe, l'intero ambiente è stato allineato a una JDK 8 reale (non solo il classpath): la generazione completa con successo (28% linee, 7% branch, mutation score nullo), confermando che il blocco precedente era interamente dovuto all'incompatibilità del runtime del tool (2021) con l'ambiente corrente.

Il **porting della suite** generata sul codice attuale fallisce però in modo conclusivo: il costruttore della versione 2.1.0 non richiede argomenti (field-injection), quello corrente ne richiede cinque (constructor-injection) – un disallineamento di API strutturale, non risolvibile senza riscrivere la suite da zero. EvoSuite resta N/A anche qui, ma per una causa diversa: l'impossibilità di portare l'output su un'API evoluta.

4.9 Evoluzione delle varianti C1–C4

A differenza di *CommandLogic*, qui lo smell corretto dal refactoring è uno solo e riguarda una singola riga (44): il logger viene istanziato sulla classe concreta (*NotificationDataBinderImpl.class*) invece che sull'interfaccia (*NotificationDataBinder.class*) – una correzione priva di qualunque effetto osservabile a runtime, cambia solo quale nome di classe compare nelle righe di log. Verificato sul sorgente di tutte e 4 le varianti: la correzione è identica e riuscita in ogni sessione indipendente.

I test forniti come l'input all'LLM di refactoring seguono la stessa scala crescente già usata per *CommandLogic* (nessuno per C1, BB per C2, BB+CF per C3, BB+CF+MT per C4) – ma qui, la scala non produce alcuna variazione osservabile: la riesecuzione delle suite originali di C0 sulle varianti (Appendice, **Tabella 21**) resta piena su ogni riga (24/24 manuali, 3/3 Randoop, 34/34 LLM), coerente con un fix meccanico a riga singola che non lascia spazio a un giudizio semantico da cui il modello possa deviare.

Confronto diretto dei quattro sorgenti: **C1 e C2 sono byte-identici** (0 differenze); C3 e C4 differiscono solo in porzioni che il compilatore scarta per costruzione (una riga vuota finale per C3; un blocco di commento di licenza duplicato per C4) – **0 differenze sul corpo del codice** in tutte e 4. Randoop esplora per riflessione sul bytecode compilato: le uniche differenze testuali tra le varianti (whitespace, commento di licenza) sono scartate dal compilatore prima di essere osservabili, quindi non c'è alcuna base logica per un risultato diverso. Gli LLM ricevono invece il sorgente come testo e vedono queste differenze, ma sono irrilevanti al compito (un commento di licenza corrotto o una riga vuota non alterano la logica del programma).

Coerentemente con questa premessa, la generazione ex novo dei test Randoop e LLM (Zero-shot, ToT) è stata eseguita una sola volta, su **C1**: Randoop (4 test curati su 35 grezzi, stesso criterio di scarto già applicato su C0), Zero-shot (12/12), Guided ToT (11/11 – include un nuovo test scritto per isolare, come caso a sé, la terza conferma indipendente dello stesso comportamento già visto su C0 via Randoop e riconfermato nello Zero-shot: *getNotificationTO()* con template nullo produce *NullPointerException*, non un risultato gestito) – **27/27**, nessuna regressione.

L'identità $C1=C2=C3=C4$ non resta solo dedotta dal diff: la stessa generazione Randoop (stesso comando, stesso seed di default), lanciata su C0 e su C1, produce sequenze **byte-per-byte identiche** – stesso numero di sequenze, stessa sequenza esatta di chiamate, stessi valori letterali. Sull'adeguatezza misurata da PIT, la stessa suite reagisce diversamente sulle due varianti: 3 mutanti uccisi su 44 su C0, contro 1 su 44 su C1. La causa è la singola riga diversa tra le due varianti (il riferimento al logger, corretto in tutte le varianti rispetto a C0): pur non alterando il comportamento osservabile della classe, è sufficiente a far reagire diversamente PIT, che genera e posiziona i propri mutanti sul bytecode reale di ciascuna variante. Il confronto completo su tutte e tre le tecniche, incluse le variazioni osservate anche su Zero-shot e ToT, è riportato in Appendice, **Tabella 22**.

I risultati di C1 vengono riutilizzati per C2/C3/C4.

4.10 Stima di reliability

Per *NotificationDataBinderImpl* l'insieme finale conta 61 test (BB 20, WB 1, MT 3, Randoop 3, Zero-shot 10, Few-shot 13, ToT 11), di cui uno escluso prima del calcolo: RT3 (Sezione 4.6), lo scenario irraggiungibile con dati realmente persistiti per il vincolo JPA *optional=false*, fuori dal profilo operativo assunto. L'insieme effettivo scende a 60.

Su questi 60 non si osserva alcun fallimento: ogni comportamento inizialmente sospetto è stato ricondotto, verificando il sorgente, a comportamento corretto (RT1/RT2 in Sezione 4.6, TF11 e RF1 in Sezione 4.3 / Sezione 4.4). Zero fallimenti su 60 test danno una reliability stimata pari a **1.0**, un valore che attesta l'assenza di difetti sui 60 scenari effettivamente esplorati, non in assoluto, sotto l'assunzione di profilo operativo uniforme già dichiarata come semplificazione.

5. Threats to Validity

Perimetro della verifica. Ogni suite discussa in questo report opera a livello di unità, con tutti i collaboratori mockati: un bug che emerge solo dall'interazione reale tra *CommandLogic/NotificationDataBinderImpl* e i loro collaboratori concreti (query DAO reali, transazioni, vincoli di schema) resta per costruzione fuori dalla portata di questo lavoro. Le righe dichiarate escluse per questo motivo (Sezione 3.1, Sezione 3.4) sono un limite di scope esplicito.

Curation e giudizio umano. I criteri di filtraggio dichiarati (Sezione 3.6, Sezione 4.6 – "una sola anomalia alla volta") richiedono un giudizio soggettivo su cosa conti come causa già osservata: un altro sperimentatore, applicando lo stesso criterio, potrebbe tracciare il confine in un punto leggermente diverso.

Convergenza cross-tecnica come evidenza, con un limite. In più punti del report, l'accordo tra tecniche indipendenti (Randoop, LLM, tester umano) è usato come segnale di robustezza di un finding – ad esempio la tripla conferma dell'NPE su *getNotificationTO()* con template nullo (Sezione 4.6, Sezione 4.9). Un caso osservato durante la generazione ex novo su *C1-CommandLogic* mostra però il limite di questo argomento: Zero-shot e Guided ToT, generati in sessioni separate senza riferimenti incrociati, hanno assunto **la stessa identica lettura sbagliata** del meccanismo di caching di *run()* – segno che una superficie di codice ambigua da lettura può produrre convergenza sull'errore. La convergenza resta un segnale utile, ma non sostituisce la verifica sul sorgente già applicata sistematicamente in questo lavoro.

Dipendenza dallo strumento LLM. Le tre tecniche di prompting sono state eseguite con un solo strumento (Microsoft 365 Copilot, modalità "Think Deeper"), in un'unica sessione per tecnica e per classe. I pattern osservati – tipi non deducibili da dipendenze escluse dal prompt, difficoltà persistenti su *ConstraintViolation/InvalidEntityException* – sono specifici a questo strumento e a questa data; non è verificato se si generalizzano ad altri modelli o a versioni future dello stesso strumento.

Limiti tecnici degli strumenti automatici. L'oracolo di regressione di Randoop si è mostrato sistematicamente più debole su metodi *void* (Sezione 4.6, test strength 20% contro il 100% su un metodo con valore di ritorno) – un limite della tecnica. Il fallimento di EvoSuite (Sezione 3.8, Sezione 4.8) è per entrambe le classi un problema di ambiente/versione del tool.

Fragilità della suite dopo il raffinamento WB/MT. I test aggiunti in Fase 2 (WB, MT) verificano sia il comportamento osservabile, in alcuni casi, sia i dettagli dell'implementazione. Questo li rende più capaci di rilevare mutanti, ma anche più esposti a rompersi per un refactoring che cambia la struttura interna senza alterare il comportamento osservabile dall'esterno – un costo diverso da quello dei test black-box della suite originale, che restano validi finché il contratto pubblico non cambia. Il progetto conserva entrambe le versioni per file separati proprio per rendere questo compromesso tracciabile: la suite BB misura stabilità comportamentale, la suite MT misura adeguatezza dei mutanti, con gradi diversi di fragilità al refactoring.

Valutazione one-shot del refactoring. Ogni variante (C1-C4) è stata generata dall'LLM di refactoring in un unico tentativo per classe; *C3-CommandLogic*, non compilante, è stato riportato come risultato empirico invece che corretto o rigenerato – coerente con l'obiettivo di misurare cosa produce realmente lo strumento in un singolo passaggio, ma non rappresentativo di cosa produrrebbe un ciclo iterativo di refactoring assistito.

Reliability. Il valore di 1.0 riportato per entrambe le classi (Sezione 3.10, Sezione 4.10) è condizionato all'assunzione di profilo operativo uniforme sull'insieme di test raccolto, una semplificazione dichiarata.

Indipendenza tra i test, per la reliability. Il calcolo $PFD = \text{fallimenti}/N$ (Sezione 3.10, Sezione 4.10) tratta i test come eventi indipendenti, è un'assunzione. Il rischio è concreto: *CommandLogic* mantiene uno stato condiviso tra invocazioni (*perContextCommands*, campo statico di *ImplementationManager*), sempre mockato in questo lavoro e quindi mai osservato influenzare un esito, ma è esattamente il tipo di variabile che, se non isolata correttamente, romperebbe l'indipendenza tra run.

6. Conclusioni

Il percorso applicato a *CommandLogic* e *NotificationDataBinderImpl* produce risultati diversi in modo coerente con la diversa natura delle due classi: il raffinamento PIT guadagna 9 punti di mutation score sulla prima, 43 sulla seconda, perché la Category Partition manuale intercetta bene i controlli condizionali ma molto meno le operazioni di copia diretta di campo, che dominano la seconda classe. Randoop resta la tecnica più economica ma strutturalmente più debole; le tre tecniche LLM raggiungono coverage nettamente superiori, con un valore aggiunto dimostrato in concreto: è un'unica asserzione, di una sola tecnica su quattro, a rilevare l'unica regressione comportamentale reale introdotta dal refactoring (*C2-CommandLogic*).

Per entrambe le classi la reliability stimata è 1.0, sotto l'assunzione dichiarata di profilo operativo uniforme – un risultato sui campioni effettivamente esplorati, non una garanzia assoluta.

Appendice: Tabelle e Figure di CommandLogic

Listato 1: pipeline di Continuous Integration su GitHub Actions

```
name: "ISW2 Tests"

on:
  push:
    branches: [ master, main ]
  pull_request:
    branches: [ master, main ]
  workflow_dispatch: # permette di rilanciarlo a mano dalla UI

jobs:
  selected-tests:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      include:
        - module: core/idrepo/logic
          test-pattern: "CommandLogic*"
          label: CommandLogic
        - module: core/provisioning-java
          test-pattern: "NotificationDataBinderImpl*"
          label: NotificationDataBinderImpl

    name: "Test ${matrix.label}"

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Java JDK
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: 25
          cache: 'maven'

      - name: Show versions
        run: |
          java -version
          mvn -version

      - name: Test ${matrix.label}
        run: >
          mvn -B -e -pl ${matrix.module} -am
          -Dtest=${matrix.test-pattern}
          -Dsurefire.failIfNoSpecifiedTests=false
          -Dmaven.build.cache.enabled=false
          -Dmodernizer.skip=true
          -DianaI.phase=none
          -Drat.skip=true
          -Dcheckstyle.skip=true
          -Dmaven.checkstyle.skip=true
          test
```

Tabella 1: *search(keyword, pageable): dimensioni e classi di equivalenza*

#	Dimensione	Tipo	Classe di equivalenza
D1	keyword	Esplicita (parametro)	Blank (null o "")

D1	keyword	Esplicita (parametro)	Valorizzata, con corrispondenza
D1	keyword	Esplicita (parametro)	Valorizzata, senza corrispondenza esatta
D2	pageable	Esplicita (parametro, ordinale)	Nominale
D2	pageable	Esplicita (parametro, ordinale)	pageSize = 0
D2	pageable	Esplicita (parametro, ordinale)	Pageable.unpaged()
D2	pageable	Esplicita (parametro, ordinale)	null
D2	pageable	Esplicita (parametro, ordinale)	Offset = dimensione risultato (limite)
D3	Stato ImplementationDAO	Ambientale (implicita)	Lista vuota
D3	Stato ImplementationDAO	Ambientale (implicita)	Lista non filtrata (N elementi)
D3	Stato ImplementationDAO	Ambientale (implicita)	Lista con N elementi (filtrata)

Tabella 2: search(...): Boundary Value Analysis su pageable

#	Parametro	Valore di frontiera	Rappresentante concreto
B1	pageNumber	-1 (sotto il minimo)	Mock Pageable: getPageNumber()=-1, getPageSize()=10
B2	pageNumber	0 (minimo valido)	PageRequest.of(0, 10)
B3	pageSize	0 (sotto il minimo)	Mock Pageable: getPageNumber()=0, getPageSize()=0
B4	pageSize	1 (minimo valido)	PageRequest.of(0, 1)
B5	offset (pageSize × pageNumber)	dimensione risultato DAO - 1 (ultimo elemento incluso)	PageRequest.of(2, 1) su DAO di 3 elementi
B6	offset	= dimensione risultato DAO (confine esatto, prima pagina vuota)	PageRequest.of(3, 1) su DAO di 3 elementi

Raggiungibilità: B1 e B3 non sono ottenibili tramite le factory pubbliche di PageRequest (che rifiutano valori sotto il minimo) e richiedono un mock diretto di Pageable.

Tabella 3: search(...): matrice dei test frame

ID	keyword	pageable	DAO (stato)	Stato del frame	Oracolo
TF1	"%backup%" (con match)	Nominale	3 elementi	Valido	O1
TF2	null (blank)	Nominale	Tutti gli elementi di tipo COMMAND, non filtrati (es. 5)	Valido	O3
TF3	"nonexistent-cmd" (nessun match)	Nominale	Lista vuota	Valido	O2
TF4	"%backup%"	Offset = 3 (limite)	3 elementi	Limite	O4
TF5	"%backup%"	pageSize = 0	3 elementi	Invalido	O4
TF6	"%backup%"	unpaged()	3 elementi	Invalido	O5
TF7	"%backup%"	null	3 elementi	Invalido	O6
TF8 (BVA)	"%backup%"	pageNumber = -1	3 elementi	Invalido	O7
TF9 (BVA)	"%backup%"	pageSize = 1 (minimo valido)	3 elementi	Valido	O1
TF10 (BVA)	"%backup%"	Offset = dimensione risultato - 1	3 elementi	Limite	O1

keyword="" non compare come frame separato: appartiene alla stessa classe di equivalenza di keyword=null (TF2), comportamento identico verificato sul contratto del DAO.

Tabella 4: search(...): legenda degli oracoli

Codice	Comportamento atteso
O1	Page<CommandTO> popolata coerentemente col DAO; count = dimensione risultato DAO; nessuna eccezione
O2	Contenuto vuoto (content.isEmpty()); count = 0; nessuna eccezione
O3	Contenuto e count pari all'intero elenco delle implementazioni di tipo COMMAND; nessuna eccezione
O4	Contenuto vuoto ma count > 0: nessuna eccezione, ma la pagina restituita non riflette il totale
O5	UnsupportedOperationException attesa – comportamento documentato di Unpaged.getPageSize()/getPageNumber() in Spring Data Commons
O6	NullPointerException attesa alla prima dereferenziazione di pageable
O7 (BVA)	Comportamento anomalo atteso, da verificare sperimentalmente: pageNumber fuori dal vincolo pubblico ≥ 0 di Pageable/PageRequest

Tabella 5: run(command): dimensioni e classi di equivalenza

#	Dimensione	Tipo	Classe di equivalenza
D1	command	Esplicita (parametro)	Nulla
D1 (key)	command.key	Esplicita (sotto-campo)	Nulla
D1 (key)	command.key	Esplicita (sotto-campo)	Valida, presente nel DAO
D1 (key)	command.key	Esplicita (sotto-campo)	Valida, assente nel DAO (<i>include "vuota"</i> – cfr. nota in Sezione 3.2)
D2	Esito ImplementationManager.build(...)	Collaborazione (implicita)	Successo
D2	Esito ImplementationManager.build(...)	Collaborazione (implicita)	Fallimento (ClassNotFoundException)
D3	command.args / esito validazione	Esplicita + Collaborazione	Nulla
D3	command.args / esito validazione	Esplicita + Collaborazione	Valido
D3	command.args / esito validazione	Esplicita + Collaborazione	Con violazioni
D4	Esito runnable.run(args)	Collaborazione (implicita)	Successo
D4	Esito runnable.run(args)	Collaborazione (implicita)	Eccezione generica

Tabella 6: run(...): matrice dei test frame

ID	command	key	build	args	runnable.run(...)	Stato	Oracolo
TF1	Non-null	Presente nel DAO	Successo	Nulla	Successo	Valido	O1
TF2	Non-null	Presente nel DAO	Successo	Valido	Successo	Valido	O1
TF3	Nulla	–	–	–	–	Invalido	O2
TF4	Non-null	Nulla	–	–	–	Invalido	O3
TF5	Non-null	Valida, assente nel DAO	–	–	–	Invalido	O3

TF6	Non-null	Presente nel DAO	Fallimento	–	–	Invalido	O4
TF7	Non-null	Presente nel DAO	Successo	Con violazioni	–	Invalido	O5
TF8	Non-null	Presente nel DAO	Successo	Valido	Eccezione	Invalido	O6

"–" indica che l'esecuzione si interrompe prima di raggiungere quella fase (es. in TF3 l'eccezione scatta alla prima dereferenziazione di command, prima di leggere key).

Tabella 7: run(...): legenda degli oracoli

Codice	Comportamento atteso
O1	Restituisce una String non nulla (il message() del Result); nessuna eccezione
O2	NullPointerException attesa alla prima dereferenziazione (command.getKey()) – nessun controllo esplicito di null sul parametro
O3	NotFoundException("Implementation " + key); nessuna costruzione né esecuzione del comando
O4	SyncopClientException di tipo InvalidImplementation, con elemento "Could not build " + key; nessuna validazione né esecuzione
O5	SyncopClientException di tipo InvalidValues, quando gli argomenti del comando violano i vincoli di Bean Validation dichiarati sulla sottoclasse di CommandArgs
O6	SyncopClientException di tipo ExecutionError, con messaggio dell'eccezione originale sollevata da run(...)

Figura 1: Report JaCoCo (solo test BB)
org.apache.syncope.core.logic

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TaskLogic		0%		0%	82	82	243	243	40	40	1	1
GroupLogic		0%		0%	42	42	180	180	29	29	1	1
AnyObjectLogic		0%		0%	39	39	146	146	27	27	1	1
ReportLogic		0%		0%	48	48	148	148	32	32	1	1
RealmLogic		0%		0%	29	29	121	121	14	14	1	1
AuditLogic		0%		0%	36	36	122	122	15	15	1	1
UserSelfLogic		0%		0%	36	36	108	108	18	18	1	1
UserLogic		0%		0%	30	30	99	99	24	24	1	1
ImplementationLogic		0%		0%	54	54	100	100	11	11	1	1
SchemaLogic		0%		0%	42	42	104	104	19	19	1	1
AbstractUserLogic		0%		0%	29	29	98	98	9	9	1	1
MfaLogic		0%		0%	30	30	93	93	18	18	1	1
AbstractAnyLogic		0%		0%	25	25	86	86	15	15	1	1
DelegationLogic		0%		0%	27	27	51	51	13	13	1	1
IdRepoLogicContext		0%		n/a	34	34	34	34	34	34	1	1
FIQLQueryLogic		0%		0%	27	27	54	54	13	13	1	1
AnyTypeLogic		0%		0%	20	20	56	56	10	10	1	1
PolicyLogic		0%		0%	17	17	44	44	10	10	1	1
MailTemplateLogic		0%		0%	25	25	58	58	14	14	1	1
SecurityQuestionLogic		0%		0%	20	20	41	41	12	12	1	1
RoleLogic		0%		0%	21	21	37	37	14	14	1	1
NotificationLogic		0%		0%	21	21	39	39	13	13	1	1
RelationshipTypeLogic		0%		0%	16	16	39	39	10	10	1	1
SyncopLogic		0%		0%	14	14	42	42	13	13	1	1
AnyTypeClassLogic		0%		0%	18	18	36	36	10	10	1	1
AbstractJobLogic		0%		0%	18	18	42	42	5	5	1	1
LogicInvocationHandler		0%		0%	9	9	44	44	3	3	1	1
AccessTokenLogic		0%		0%	11	11	33	33	10	10	1	1
CommandLogic		74%		100%	5	13	14	57	5	10	0	1
AbstractLogic		26%		0%	3	5	4	6	1	3	0	1
AbstractTransactionalLogic.ProvisioningInfo		0%		n/a	1	1	1	1	1	1	1	1
AbstractJobLogic.JobReference		0%		n/a	1	1	1	1	1	1	1	1
UserSelfLogic.Self		0%		n/a	1	1	1	1	1	1	1	1
AbstractAnyLogic.BeforeResult		0%		n/a	1	1	1	1	1	1	1	1
UnresolvedReferenceException		0%		n/a	2	2	4	4	2	2	1	1
AbstractExecutableLogic		0%		n/a	1	1	2	2	1	1	1	1
AbstractTransactionalLogic		0%		n/a	1	1	1	1	1	1	1	1
Total	11,555 of 11,751	1%	712 of 718	0%	836	846	2,327	2,372	469	476	35	37

CommandLogic

Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Ctxy	Missed	Lines	Missed	Methods
read(String)		0%		n/a	1	1	8	8	1	1
lambda\$search\$0(Implementation)		57%		n/a	0	1	3	5	0	1
lambda\$run\$2(Implementation_Command)		0%		n/a	1	1	1	1	1	1
lambda\$run\$1(Implementation)		0%		n/a	1	1	1	1	1	1
lambda\$read\$0(String)		0%		n/a	1	1	1	1	1	1
resolveReference(Method_Object[])		0%		n/a	1	1	1	1	1	1
run(CommandTO)		100%		100%	0	4	0	27	0	1
search(String_Pageable)		100%		n/a	0	1	0	9	0	1
CommandLogic(ImplementationDAO_Validator)		100%		n/a	0	1	0	5	0	1
lambda\$run\$0(CommandTO)		100%		n/a	0	1	0	1	0	1
Total	66 of 255	74%	0 of 6	100%	5	13	14	57	5	10

Figura 2: dettaglio di miss (solo test BB)

```

56.     protected final Map<String, Command?>> perContextCommands = new ConcurrentHashMap<>();
57.
58.     public CommandLogic(final ImplementationDAO implementationDAO, final Validator validator) {
59.         this.implementationDAO = implementationDAO;
60.         this.validator = validator;
61.     }
62.
63.     @PreAuthorize("hasRole('" + IdRepoEntitlement.IMPLEMENTATION_LIST + "')")
64.     @Transactional(readOnly = true)
65.     public Page<CommandTO> search(final String keyword, final Pageable pageable) {
66.         List<Implementation> result = implementationDAO.findByTypeAndKeyword(IdRepoImplementationType.COMMAND, keyword);
67.
68.         long count = result.size();
69.
70.         List<CommandTO> commands = result.stream().
71.             skip(pageable.getPageSize() * pageable.getPageNumber()).
72.             limit(pageable.getPageSize()).
73.             map(command -> {
74.                 try {
75.                     return new CommandTO.Builder(command.getKey()).
76.                         args(ImplementationManager.emptyArgs(AuthContextUtils.getDomain(), command)).build();
77.                 } catch (Exception e) {
78.                     LOG.error("Could not get arg class for {}", command, e);
79.                     return null;
80.                 }
81.             }).
82.             filter(Objects::nonNull).
83.             toList();
84.
85.         return new SyncopePage<>(commands, pageable, count);
86.     }

```

Figura 3: Report JaCoCo (BB, WB)
org.apache.syncope.core.logic

Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Ctxy	Missed	Lines	Missed	Methods	Missed	Classes
TaskLogic		0%		0%	82	82	243	243	40	40	1	1
GroupLogic		0%		0%	42	42	180	180	29	29	1	1
AnyObjectLogic		0%		0%	39	39	146	146	27	27	1	1
ReportLogic		0%		0%	48	48	148	148	32	32	1	1
RealmLogic		0%		0%	29	29	121	121	14	14	1	1
AuditLogic		0%		0%	36	36	122	122	15	15	1	1
UserSelfLogic		0%		0%	36	36	108	108	18	18	1	1
UserLogic		0%		0%	30	30	99	99	24	24	1	1
ImplementationLogic		0%		0%	54	54	100	100	11	11	1	1
SchemaLogic		0%		0%	42	42	104	104	19	19	1	1
AbstractUserLogic		0%		0%	29	29	98	98	9	9	1	1
MfaLogic		0%		0%	30	30	93	93	18	18	1	1
AbstractAnyLogic		0%		0%	25	25	86	86	15	15	1	1
DelegationLogic		0%		0%	27	27	51	51	13	13	1	1
IdRepoLogicContext		0%		n/a	34	34	34	34	34	34	1	1
FIQLQueryLogic		0%		0%	27	27	54	54	13	13	1	1
AnyTypeLogic		0%		0%	20	20	56	56	10	10	1	1
PolicyLogic		0%		0%	17	17	44	44	10	10	1	1
MailTemplateLogic		0%		0%	25	25	58	58	14	14	1	1
SecurityQuestionLogic		0%		0%	20	20	41	41	12	12	1	1
RoleLogic		0%		0%	21	21	37	37	14	14	1	1
NotificationLogic		0%		0%	21	21	39	39	13	13	1	1
RelationshipTypeLogic		0%		0%	16	16	39	39	10	10	1	1
SyncopeLogic		0%		0%	14	14	42	42	13	13	1	1
AnyTypeClassLogic		0%		0%	18	18	36	36	10	10	1	1
AbstractJobLogic		0%		0%	18	18	42	42	5	5	1	1
LogicInvocationHandler		0%		0%	9	9	44	44	3	3	1	1
AccessTokenLogic		0%		0%	11	11	33	33	10	10	1	1
CommandLogic		77%		100%	5	13	11	57	5	10	0	1
AbstractLogic		26%		0%	3	5	4	6	1	3	0	1
AbstractTransactionalLogic.ProvisioningInfo		0%		n/a	1	1	1	1	1	1	1	1
AbstractJobLogic.JobReference		0%		n/a	1	1	1	1	1	1	1	1
UserSelfLogic.Self		0%		n/a	1	1	1	1	1	1	1	1
AbstractAnyLogic.BeforeResult		0%		n/a	1	1	1	1	1	1	1	1
UnresolvedReferenceException		0%		n/a	2	2	4	4	2	2	1	1
AbstractExecutableLogic		0%		n/a	1	1	2	2	1	1	1	1
AbstractTransactionalLogic		0%		n/a	1	1	1	1	1	1	1	1
Total	11,547 of 11,751	1%	712 of 718	0%	836	846	2,324	2,372	469	476	35	37

Esempio 1: candidato bug trovato su seach()

Campo	Valore
Contesto	RF1, CommandLogicWBTest, raffinamento guidato da JaCoCo su search(...) (righe 77-79)
Comando	mvn test -pl core/idrepo/logic -Dtest=CommandLogicWBTest#rf1_oneElementFailsToBuild
Scenario	DAO con 3 elementi; ImplementationManager.emptyArgs(...) fallisce per il secondo
Esito osservato	page.getContent().size() = 2 (elemento fallace filtrato in silenzio); page.getTotalElements() = 3 (invariato)
Causa isolata	count = result.size() calcolato sulla lista grezza del DAO, prima dello stream .map(...).filter(Objects::nonNull) che rimuove le conversioni fallite
Stato	Candidato bug – comportamento non documentato; RF1 lo osserva e lo asserisce esplicitamente

```
76 @Test
77 @DisplayName("RF1: DAO con un elemento che fallisce la build -> content=2 ma count=3")
78 void rf1_oneElementFailsToBuild() {
79     // Arrange
80     Implementation impl1 = mockImplementation( key: "cmd1");
81     Implementation impl2 = mockImplementation( key: "cmd2");
82     Implementation impl3 = mockImplementation( key: "cmd3");
83     when(implementationDAO.findByTypeAndKeyword(IdRepoImplementationType.COMMAND, keywords: "%backup%")).
84         thenReturn(List.of(impl1, impl2, impl3));
85
86     try (MockedStatic<ImplementationManager> im = mockStatic(ImplementationManager.class)) {
87         im.when(() -> ImplementationManager.emptyArgs(anyString(), eq(impl1))).
88             thenReturn(new CommandArgs());
89         im.when(() -> ImplementationManager.emptyArgs(anyString(), eq(impl2))).
90             thenThrow(new RuntimeException("boom"));
91         im.when(() -> ImplementationManager.emptyArgs(anyString(), eq(impl3))).
92             thenReturn(new CommandArgs());
93
94         // Act
95         Page<CommandTO> page = commandLogic.search( keyword: "%backup%", PageRequest.of( pageNumber: 0, pageSize: 10));
96
97         // Assert
98         assertEquals( expected: 2, page.getContent().size());
99         assertEquals( expected: 3, page.getTotalElements());
100     }
101 }
102 }
```

Figura 4: dettaglio su righe non coperte (BB,WB)

```
104. @PreAuthorize("hasRole('" + IdRepoEntitlement.COMMAND_RUN + "')")
105. @SuppressWarnings("unchecked")
106. public String run(final CommandTO command) {
107.     Implementation impl = implementationDAO.findById(command.getKey()).
108.         orElseThrow(() -> new NotFoundException("Implementation " + command.getKey()));
109.
110.     Command<CommandArgs> runnable;
111.     try {
112.         runnable = (Command<CommandArgs>) ImplementationManager.build(
113.             AuthContextUtils.getDomain(),
114.             impl,
115.             () -> perContextCommands.get(impl.getKey()),
116.             instance -> perContextCommands.put(impl.getKey(), instance));
117.     } catch (Exception e) {
118.         SyncopClientException sce = SyncopClientException.build(ClientExceptionType.InvalidImplementation);
119.         sce.getElements().add("Could not build " + impl.getKey());
120.         throw sce;
121.     }
122. }
```

Figura 5: Report PIT (BB,WB)
Pit Test Coverage Report

Package Summary

org.apache.syncope.core.logic

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	81% 46/57	64% 7/11	88% 7/8

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CommandLogic.java	81% 46/57	64% 7/11	88% 7/8

Report generated by PIT 1.25.8 [support](#)

Mutations

71	1. Replaced integer multiplication with division → KILLED
75	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$search\$0 → KILLED
85	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::search → KILLED
92	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$read\$0 → NO_COVERAGE
95	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::read → NO_COVERAGE
108	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$run\$0 → KILLED
115	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$run\$1 → NO_COVERAGE
123	1. negated conditional → KILLED
126	1. negated conditional → KILLED
139	1. negated conditional → SURVIVED Covering tests
	2. replaced return value with "" for org/apache/syncope/core/logic/CommandLogic::run → KILLED

Figura 6: Report PIT (BB,WB,MT)
Pit Test Coverage Report

Package Summary

org.apache.syncope.core.logic

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	81% 46/57	73% 8/11	100% 8/8

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CommandLogic.java	81% 46/57	73% 8/11	100% 8/8

Report generated by PIT 1.25.8 [support](#)

Mutations

71	1. Replaced integer multiplication with division → KILLED
75	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$search\$0 → KILLED
85	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::search → KILLED
92	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$read\$0 → NO_COVERAGE
95	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::read → NO_COVERAGE
108	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$run\$0 → KILLED
115	1. replaced return value with null for org/apache/syncope/core/logic/CommandLogic::lambda\$run\$1 → NO_COVERAGE
123	1. negated conditional → KILLED
126	1. negated conditional → KILLED
139	1. negated conditional → KILLED
	2. replaced return value with "" for org/apache/syncope/core/logic/CommandLogic::run → KILLED

Figura 7: Report JaCoCo (Randoop)
CommandLogic

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
run(CommandTO)		5%		0%	3 4	26 27	0 1
read(String)		0%		n/a	1 1	8 8	1 1
lambda\$search\$0(Implementation)		0%		n/a	1 1	5 5	1 1
lambda\$run\$2(Implementation, Command)		0%		n/a	1 1	1 1	1 1
lambda\$run\$1(Implementation)		0%		n/a	1 1	1 1	1 1
lambda\$read\$0(String)		0%		n/a	1 1	1 1	1 1
resolveReference(Method, Object[])		0%		n/a	1 1	1 1	1 1
search(String, Pageable)		100%		n/a	0 1	0 9	0 1
CommandLogic(ImplementationDAO, Validator)		100%		n/a	0 1	0 5	0 1
lambda\$run\$0(CommandTO)		100%		n/a	0 1	0 1	0 1
Total	191 of 255	25%	6 of 6	0%	9 13	41 57	6 10

```

56.     protected final Map<String, Command<?>> perContextCommands = new ConcurrentHashMap<>();
57.
58.     public CommandLogic(final ImplementationDAO implementationDAO, final Validator validator) {
59.         this.implementationDAO = implementationDAO;
60.         this.validator = validator;
61.     }
62.
63.     @PreAuthorize("hasRole('" + IdRepoEntitlement.IMPLEMENTATION_LIST + "')")
64.     @Transactional(readonly = true)
65.     public Page<CommandTO> search(final String keyword, final Pageable pageable) {
66.         List<Implementation> result = implementationDAO.findByTypeAndKeyword(IdRepoImplementationType.COMMAND, keyword);
67.
68.         long count = result.size();
69.
70.         List<CommandTO> commands = result.stream().
71.             skip(pageable.getPageSize() * pageable.getPageNumber()).
72.             limit(pageable.getPageSize()).
73.             map(command -> {
74.                 try {
75.                     return new CommandTO.Builder(command.getKey()).
76.                         args(ImplementationManager.emptyArgs(AuthContextUtils.getDomain(), command)).build();
77.                 } catch (Exception e) {
78.                     LOG.error("Could not get arg class for {}", command, e);
79.                     return null;
80.                 }
81.             }).
82.             filter(Objects::nonNull).
83.             toList();
84.
85.         return new SyncopePage<>(commands, pageable, count);
86.     }
104.
105.     @PreAuthorize("hasRole('" + IdRepoEntitlement.COMMAND_RUN + "')")
106.     @SuppressWarnings("unchecked")
107.     public String run(final CommandTO command) {
108.         Implementation impl = implementationDAO.findById(command.getKey());
109.         orElseThrow(() -> new NotFoundException("Implementation " + command.getKey()));
110.
111.         Command<CommandArgs> runnable;
112.         try {
113.             runnable = (Command<CommandArgs>) ImplementationManager.build(
114.                 AuthContextUtils.getDomain(),
115.                 impl,
116.                 () -> perContextCommands.get(impl.getKey()),
117.                 instance -> perContextCommands.put(impl.getKey(), instance));
118.         } catch (Exception e) {
119.             SyncopeClientException sce = SyncopeClientException.build(ClientExceptionType.InvalidImplementation);
120.             sce.getElements().add("Could not build " + impl.getKey());
121.             throw sce;
122.         }
123.
124.         if (command.getArgs() != null) {
125.             try {
126.                 Set<ConstraintViolation<Object>> violations = validator.validate(command.getArgs());
127.                 if (!violations.isEmpty()) {
128.                     throw new InvalidEntityException(command.getArgs().getClass().getName(), violations);
129.                 }
130.             } catch (ValidationException e) {
131.                 LOG.error("While validating {}", command.getArgs(), e);
132.             }
133.
134.             SyncopeClientException sce = SyncopeClientException.build(ClientExceptionType.InvalidValues);
135.             sce.getElements().add(e.getMessage());
136.             throw sce;
137.         }
138.
139.         try {
140.             return runnable.run(command.getArgs() == null
141.                 ? ImplementationManager.emptyArgs(AuthContextUtils.getDomain(), impl)
142.                 : command.getArgs()).message();
143.         } catch (Exception e) {
144.             LOG.error("While running {} on {}", command.getKey(), command.getArgs(), e);
145.
146.             SyncopeClientException sce = SyncopeClientException.build(ClientExceptionType.ExecutionError);
147.             sce.getElements().add(e.getMessage());
148.             throw sce;
149.         }

```

Prompt 1: LLM Zero-shot

As a professional software tester who writes Java test methods, generate a complete JUnit 5 test file (CommandLogicLLMZeroShotTest.java) to test the following methods of the class named CommandLogic:

- search(String keyword, Pageable pageable)
- run(CommandTO command)

Your output file must start with **###Test START##** and finish with **###Test END##**.

Prompt 2: LLM Few-shot

As a professional software tester who writes Java test methods, consider the following examples:

```
@Test
@DisplayName("TF1: keyword con wildcard/match, pageable nominale, 3 elementi -> pagina popolata (O1)")
void tf1_nominal() {
    // Arrange
    Implementation impl1 = mockImplementation("cmd1");
    Implementation impl2 = mockImplementation("cmd2");
    Implementation impl3 = mockImplementation("cmd3");
    when(implementationDAO.findByTypeAndKeyword(IdRepoImplementationType.COMMAND, "%backup%")).
        thenReturn(List.of(impl1, impl2, impl3));

    try (MockedStatic<ImplementationManager> im = mockStatic(ImplementationManager.class)) {
        im.when(() -> ImplementationManager.emptyArgs(anyString(), any(Implementation.class))).
            thenReturn(new CommandArgs());

        // Act
        Page<CommandTO> page = commandLogic.search("%backup%", PageRequest.of(0, 10));

        // Assert
        assertEquals(3, page.getContent().size());
        assertEquals(3, page.getTotalElements());
        assertEquals(List.of("cmd1", "cmd2", "cmd3"),
            page.getContent().stream().map(CommandTO::getKey).toList());
    }
}

@Test
@DisplayName("TF7: args con violazioni di vincolo -> SyncopClientException InvalidValues (O5)")
void tf7_argsWithViolations() {
    // Arrange
    Implementation impl = mockImplementation("clear-cache");
    doReturn(Optional.of(impl)).when(implementationDAO).findById("clear-cache");
    ConstraintViolation<Object> violation = mockViolation();
    when(validator.validate(any())).thenReturn(Set.of(violation));

    @SuppressWarnings("unchecked")
    Command<CommandArgs> runnable = mock(Command.class);

    CommandTO command = new CommandTO.Builder("clear-cache").args(new CommandArgs()).build();

    try (MockedStatic<ImplementationManager> im = mockStatic(ImplementationManager.class)) {
        im.when(() -> ImplementationManager.build(anyString(), eq(impl), any(), any())).
            thenReturn(runnable);
    }
}
```

```
// Act
SyncopeClientException sce = assertThrows(SyncopeClientException.class,
    () -> commandLogic.run(command));

// Assert
assertEquals(ClientExceptionType.InvalidValues, sce.getType());
}
}
```

Now, generate a complete JUnit 5 test file (CommandLogicLLMFewShotTest.java) to test the following methods of the class named CommandLogic:

- search(String keyword, Pageable pageable)
- run(CommandTO command)

Prompt 3: LLM Guided ToT

Imagine three different experts in software testing who are tasked with developing comprehensive JUnit 5 test cases for the following Java class. To comprehensively test the following methods of the class named CommandLogic:

- search(String keyword, Pageable pageable)
- run(CommandTO command)

they must following these steps:

- Extract and list the signatures of the two methods above
- For each method, generate a basic JUnit 5 test case that checks the method's functionality
- Given the source code of the class and the listed methods, identify potential edge cases and exception handling scenarios that should be tested
- Generate JUnit 5 test cases that specifically test for the identified edge cases and exceptions
- Merge all the individual test cases into a complete JUnit 5 test file CommandLogicLLMToTTest.java

Tabella 8: LLM su CommandLogic, coverage per tecnica (singola)

Tecnica	Istruzioni	Branch
Zero-shot	77%	100%
Few-shot	77%	100%
ToT	77%	100%

Tabella 9: LLM su CommandLogic, sintesi per tecnica

Metrica	Zero-shot	Few-shot	ToT
Test generati	9	12	13
Errori di compilazione iniziali	7	0	7
Riparati correttamente da Copilot	1 (per aggiramento)	2 problemi su 3 cause	1 causa (compilazione)
Bug introdotto dalla correzione stessa	0	0	1
Riparati manualmente	7	1	3
Scenari nuovi	2/9 (vs. manuale)	2/12 (vs. manuale), 1/12 (vs. zero-shot)	2/13 (vs. tutte le tecniche)

Figura 8: Report JaCoCo (LLM Zero-shot, Few-shot, Guided ToT)

CommandLogic

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● read(String)		0%		n/a	1	1	8	8	1	1
● lambda\$run\$2(Implementation, Command)		0%		n/a	1	1	1	1	1	1
● lambda\$run\$1(Implementation)		0%		n/a	1	1	1	1	1	1
● lambda\$read\$0(String)		0%		n/a	1	1	1	1	1	1
● resolveReference(Method, Object[])		0%		n/a	1	1	1	1	1	1
● run(CommandTO)		100%		100%	0	4	0	27	0	1
● search(String, Pageable)		100%		n/a	0	1	0	9	0	1
● lambda\$search\$0(Implementation)		100%		n/a	0	1	0	5	0	1
● CommandLogic(ImplementationDAO, Validator)		100%		n/a	0	1	0	5	0	1
● lambda\$run\$0(CommandTO)		100%		n/a	0	1	0	1	0	1
Total	58 of 255	77%	0 of 6	100%	5	13	11	57	5	10

Listato 2: CommandLogicLLMZeroShotRaw.java, metodo `searchReturnsRequestedPageWithCommandTransferObjects` (output grezzo, non corretto)

```

59     @Test
60     void searchReturnsRequestedPageWithCommandTransferObjects() {
61         Implementation first = implementation("command-1");
62         Implementation second = implementation("command-2");
63         Implementation third = implementation("command-3");
64
65         CommandArgs argsForSecond = mock(CommandArgs.class);
66         CommandArgs argsForThird = mock(CommandArgs.class);
67
68         when(implementationDAO.findByIdAndKeyword(IdRepoImplementationType.COMMAND, "cmd")).
69             thenReturn(List.of(first, second, third));
70
71         try (MockedStatic<AuthContextUtils> authContextUtils = Mockito.mockStatic(AuthContextUtils.class);
72              MockedStatic<ImplementationManager> implementationManager =
73                  Mockito.mockStatic(ImplementationManager.class)) {
74
75             authContextUtils.when(AuthContextUtils::getDomain).thenReturn(DOMAIN);
76             implementationManager.when(() -> ImplementationManager.emptyArgs(DOMAIN, second)).
77                 thenReturn(argsForSecond);
78             implementationManager.when(() -> ImplementationManager.emptyArgs(DOMAIN, third)).
79                 thenReturn(argsForThird);
80
81             Page<CommandTO> result = logic.search("cmd", PageRequest.of(1, 2));
82
83             assertEquals(3L, result.getTotalElements());
84             assertEquals(2, result.getNumber());
85             assertEquals(2, result.getSize());
86             assertEquals(2, result.getContent().size());
87             assertEquals("command-2", result.getContent().get(0).getKey());
88
89             assertEquals(argsForSecond, result.getContent().get(0).getArgs());
90             assertEquals("command-3", result.getContent().get(1).getKey());
91             assertEquals(argsForThird, result.getContent().get(1).getArgs());
92
93             implementationManager.verify(() -> ImplementationManager.emptyArgs(DOMAIN, second));
94             implementationManager.verify(() -> ImplementationManager.emptyArgs(DOMAIN, third));
95         }
96     }

```

Errore: `PageRequest.of(1,2)` con 3 elementi totali nel DAO produce, nell'implementazione reale di `search(...)`, un solo elemento in pagina, non due come previsto qui in `assertEquals(2, result.getContent().size())` e nei due elementi asseriti (`command-2`, `command-3`).

Figura 9: errore di compilazione dovuto dal wildcard

```

125     @Test
126     @SuppressWarnings("unchecked")
127     void runBuildsCommandValidatesProvidedArgsAndReturnsMessage() throws Exception {
128         Implementation implementation = implementation("echo");
129         CommandArgs commandArgs = mock(CommandArgs.class);
130         CommandT0 commandT0 = new CommandT0.Builder("echo").args(commandArgs).build();
131         Command<CommandArgs> runnable = mock(Command.class);
132
133         when(implementationDAO.findById("echo")).thenReturn(Optional.of(implementation));
134         when(validator.validate(commandArgs)).thenReturn(Set.of());
135         when(runnable.run(commandArgs)).
136             thenReturn(new Command.Result("executed", Map.<String, Serializable>of()));

```

Figura 10: un esempio di errore AssertionError, per il tipo errato

```

259     @Test
260     @DisplayName("TF9: build del comando fallisce -> SyncopClientException InvalidImplementation (07)
261     void tf9_runBuildFails() {
262         // Arrange
263         Implementation impl = mockImplementation("broken");
264         doReturn(Optional.of(impl)).when(implementationDAO).findById("broken");
265
266         CommandT0 command = new CommandT0.Builder("broken").args(new CommandArgs()).build();
267
268         try (MockedStatic<AuthContextUtils> acu = mockStatic(AuthContextUtils.class);
269             MockedStatic<ImplementationManager> im = mockStatic(ImplementationManager.class)) {
270
271             acu.when(AuthContextUtils::getDomain).thenReturn(DOMAIN);
272             im.when(() -> ImplementationManager.build(eq(DOMAIN), eq(impl), any(), any()))
273                 .thenReturn(new IllegalStateException("Cannot build command"));
274
275             // Act
276             SyncopClientException sce = assertThrows(SyncopClientException.class,
277                 () -> commandLogic.run(command));
278
279             // Assert
280             assertEquals(ClientExceptionType.InvalidImplementation, sce.getType());
281             assertEquals(List.of("Could not build broken"), sce.getElements());
282         }
283     }

```

Figura 11: correzione con aggiunta di lenient() in Few-shot test

```

16 usages  ▲ valejin
385 @ private Implementation mockImplementation(final String key) {
386     Implementation implementation = mock(Implementation.class);
387     Mockito.lenient().when(implementation.getKey()).thenReturn(key);
388     return implementation;
389 }

```

Esempio 2: bug su stub di ConstraintViolation

Campo	Valore
Contesto	Bug ricorrente sullo stub di ConstraintViolation, presentato a Copilot 4 volte su 3 tecniche
Scenario	Mock con solo getMessage() stubbato; il costruttore reale di InvalidEntityException legge invece getMessageTemplate(), getPropertyPath(), getInvalidValue(), getLeafBean()
Esito osservato	NullPointerException dentro InvalidEntityException.<init>
Causa isolata	Pattern-matching sul nome di metodo più plausibile, senza tracciare l'esecuzione reale di codice di terze parti mai incluso nei prompt
Correzione	Manuale, diagnosi diretta sul sorgente; mai risolto autonomamente da Copilot in 4/4 tentativi

Tabella 10: (CommandLogic) test forniti in input all'LLM per la generazione delle varianti, se passano ancora i test della classe C0

Classe	Versione	Test forniti in prompt	Compila	Test manuali	Test Randoop	Test LLM
C0	Originale	— (baseline)	Sì	21/21	3/3	34/34
C1	Refactored	Nessuno	Sì	21/21	3/3	34/34
C2	Refactored	BB	Sì	21/21	3/3	30/34*
C3	Refactored	BB + CF (JaCoCo/WB)	No	—	—	—
C4	Refactored	BB + CF + MT	Sì	21/21	3/3	34/34

*La variante C2 ha avuto due failure sui test Few-shot di C0 e due failure sui test Zero-shot di C0.

Esempio 3: finding della varainte C2 di CommandLogic

Campo	Valore
Contesto	LLM Zero-shot su C2, unico finding che rileva la regressione
Esito osservato	assertTrue(exception.getElements().contains("boom")) fallisce
Causa isolata	C2 ha rimosso sce.getElements().add(e.getMessage()) dal catch finale – comportamento realmente cambiato
Stato	Finding reale; test marcato @Disabled con motivazione (fallimento atteso, richiesto per eseguire PIT su suite verde), non corretto né lasciato semplicemente fallito.

```

159  @Test
160  @Disabled("Fallimento atteso: C2 ha rimosso sce.getElements().add(e.getMessage()) dal "
161    + "catch finale di run(). "
162    + "Disabilitato solo per consentire l'analisi di mutation testing (PIT richiede una "
163    + "suite verde). La regressione resta documentata e verificata "
164    + "manualmente, non e' stata 'corretta' indebolendo l'asserzione.")
165  void runWrapsCommandExecutionExceptionAsSyncopeClientException() throws Exception {
166      Implementation implementation = implementation(KEY_ONE);
167      CommandArgs args = new CommandArgs();
168      CommandT0 commandT0 = new CommandT0.Builder(KEY_ONE).args(args).build();
169
170      /unchecked/
171      Command<CommandArgs> runnable = mock(Command.class);
172
173      doReturn(Optional.of(implementation)).when(implementationDAO).findById(KEY_ONE);
174      when(validator.validate(args)).thenReturn(Set.of());
175      when(runnable.run(args)).thenReturn(new IllegalStateException("boom"));
176
177      try (MockedStatic<ImplementationManager> ignored = mockImplementationBuild(implementation, runnable)) {
178          SyncopeClientException exception =
179              assertThrows(SyncopeClientException.class, () -> logic.run(commandT0));
180
181          assertTrue(exception.getElements().contains("boom"));
182          verify(runnable).run(args);
183      }
184  }

```

Tabella 11: Comparativa Randoop/Zero-shot/ToT, CommandLogic C0/C1/C2

Suite	Variante	Istruzioni	Branch	Line Cov. (PIT)	Mutation Cov.	Test Strength
Randoop (curato)	C0	25%	0%	28%	27% (3/11)	100% (3/3)
Randoop (curato, 12 test)	C1	25,1%	0%	28%	27% (3/11)	100% (3/3)
Randoop (curato)	C2	28%	0%	29%	27% (3/11)	100% (3/3)
Zero-shot (singola)	C0	77%	100%	81% (46/57)	73% (8/11)	100% (8/8)
Zero-shot (singola)	C1	75,7%	66,7%	79% (45/57)	73% (8/11)	100% (8/8)
Zero-shot (singola)	C2	75%	100%	73% (40/55)	73% (8/11)	100% (8/8)
ToT (singola)	C0	77%	100%	81% (46/57)	73% (8/11)	100% (8/8)
ToT (singola)	C1	77,3%	100%	81% (46/57)	73% (8/11)	100% (8/8)
ToT (singola)	C2	75%	100%	80% (44/55)	73% (8/11)	100% (8/8)

Tabella 12: Delta comparativa rispetto a C0, CommandLogic

Suite	Variante	Δ Istruzioni	Δ Branch	Δ Line Cov. (PIT)	Δ Mutation (PIT)	Mutation assoluto
Randoop (curato)	C1	+0,1%	0%	0% (28%→28%)	0%	27% (3/11)
Randoop (curato)	C2	+3%	0%	+1% (28%→29%)	0%	27% (3/11)
Zero-shot (singola)	C1	-1,3%	-33,3%	-2% (81%→79%)	0%	73% (8/11)
Zero-shot (singola)	C2	-2%	0%	-8% (81%→73%)	0%	73% (8/11)
ToT (singola)	C1	+0,3%	0%	0% (81%→81%)	0%	73% (8/11)
ToT (singola)	C2	-2%	0%	-1% (81%→80%)	0%	73% (8/11)

Appendice: Tabelle e Figure di NotificationDataBinderImpl

Tabella 13: update(...): dimensioni e classi di equivalenza

#	Dimensione	Tipo	Classe di equivalenza
D1	notification	Esplicita (parametro)	Nulla
D1	notification	Esplicita (parametro)	Non nulla (mock valido)
D2.1	notificationTO.template	Esplicita (sotto-campo)	Con corrispondenza in mailTemplateDAO
D2.1	notificationTO.template	Esplicita (sotto-campo)	Senza corrispondenza
D2.1	notificationTO.template	Esplicita (sotto-campo)	Nulla
D2.2	notificationTO.events	Esplicita (sotto-campo)	Vuota
D2.2	notificationTO.events	Esplicita (sotto-campo)	Non vuota
D2.3	notificationTO.staticRecipients	Esplicita (sotto-campo)	Vuota
D2.3	notificationTO.staticRecipients	Esplicita (sotto-campo)	Indirizzi conformi a EMAIL_PATTERN
D2.3	notificationTO.staticRecipients	Esplicita (sotto-campo)	Almeno un indirizzo non conforme
D2.4	notificationTO.abouts	Esplicita (sotto-campo)	Vuota
D2.4	notificationTO.abouts	Esplicita (sotto-campo)	Chiavi risolvibili, valori non-blank
D2.4	notificationTO.abouts	Esplicita (sotto-campo)	Almeno una chiave non risolvibile
D2.4	notificationTO.abouts	Esplicita (sotto-campo)	Almeno un valore blank
D2.5	notificationTO.recipientAttrName	Esplicita (sotto-campo)	Parsabile
D2.5	notificationTO.recipientAttrName	Esplicita (sotto-campo)	Non parsabile
D2.5	notificationTO.recipientAttrName	Esplicita (sotto-campo)	Nulla
D2.6	notificationTO.recipientsProvider	Esplicita (sotto-campo)	Nulla
D2.6	notificationTO.recipientsProvider	Esplicita (sotto-campo)	Con corrispondenza in implementationDAO
D2.6	notificationTO.recipientsProvider	Esplicita (sotto-campo)	Senza corrispondenza
D3	Stato iniziale notification.getAbouts()	Ambientale (implicita)	Coincidente con le chiavi di notificationTO.abouts
D3	Stato iniziale notification.getAbouts()	Ambientale (implicita)	Elemento per un AnyType non presente in notificationTO.abouts

Tabella 14: update(...): matrice dei test frame

ID	Variazione	Stato	Oracolo
TF1	Tutti i campi al valore nominale (template con corrispondenza, events non vuota, staticRecipients con indirizzo conforme, abouts con chiave risolvibile e valore non-blank, recipientAttrName parsabile, recipientsProvider con corrispondenza)	Valido	O1
TF2	notification = null; resto nominale	Invalido	O2
TF3	template nullo o senza corrispondenza; resto nominale	Invalido	O3 (elemento "template")
TF4	events vuota; resto nominale	Invalido	O3 (elemento "events")
TF5	staticRecipients con un indirizzo non conforme; resto nominale	Invalido	O3 (elemento "staticRecipients: ...")
TF6	recipientAttrName non parsabile; resto nominale	Invalido	O4
TF7	abouts con una chiave non risolvibile; resto nominale	Limite	O5
TF8	abouts con un valore blank; resto nominale	Limite	O5
TF9	recipientsProvider valorizzato senza corrispondenza; resto nominale	Limite	O6
TF10	recipientsProvider esplicitamente nullo; resto nominale	Valido/Limite	O6
TF11	Stato iniziale di notification.getAbouts() con un elemento per un AnyType non presente in notificationTO.getAbouts(); resto nominale	Limite	O7

Tabella 15: update(...): legenda degli oracoli

Codice	Comportamento atteso
O1	Nessuna eccezione; notification aggiornata coerentemente con i valori sintatticamente validi di notificationTO
O2 (ipotesi)	Eccezione attesa per notification nullo – comportamento non documentato, da verificare sperimentalmente
O3	SyncopClientException di tipo RequiredValuesMissing – l'elemento riportato varia per test frame ("template" / "events" / "staticRecipients: ...")
O4	SyncopClientException di tipo InvalidRequest
O5	Nessuna eccezione; l'elemento anomalo di abouts (chiave non risolvibile o valore blank) viene ignorato, il resto dell'entità aggiornato normalmente
O6	Nessuna eccezione; recipientsProvider gestito senza eccezione sia quando non risolvibile sia quando esplicitamente nullo
O7 (ipotesi)	Coerentemente con una semantica di sostituzione completa, l'about non più presente in notificationTO viene rimosso dall'entità – da verificare sperimentalmente

Nota sulla struttura: TF3/TF4/TF5 condividono lo stesso oracolo O3 (stesso tipo di eccezione, RequiredValuesMissing) ma con elemento riportato diverso.

Tabella 16: getNotificationTO(...): dimensioni e classi di equivalenza

#	Dimensione	Tipo	Classe di equivalenza
D1.1	notification.template	Esplicita (sotto-campo)	Non nullo (con key)
D1.1	notification.template	Esplicita (sotto-campo)	Nullo
D1.2	notification.events	Esplicita (sotto-campo)	Non nulla, non vuota
D1.2	notification.events	Esplicita (sotto-campo)	Non nulla, vuota
D1.2	notification.events	Esplicita (sotto-campo)	Nulla

D1.3	notification.staticRecipients	Esplicita (sotto-campo)	Non nulla, non vuota
D1.3	notification.staticRecipients	Esplicita (sotto-campo)	Non nulla, vuota
D1.3	notification.staticRecipients	Esplicita (sotto-campo)	Nulla
D1.4	notification.abouts	Esplicita (sotto-campo)	Vuota
D1.4	notification.abouts	Esplicita (sotto-campo)	AnyType annidato non nullo
D1.4	notification.abouts	Esplicita (sotto-campo)	Almeno un AnyType annidato nullo
D1.5	notification.recipientsProvider	Esplicita (sotto-campo)	Nulla
D1.5	notification.recipientsProvider	Esplicita (sotto-campo)	Non nullo (con key)

Tabella 17: getNotificationTO(...): matrice dei test frame

ID	Variazione	Stato	Oracolo
TF1	Tutti i campi valorizzati coerentemente (template non nullo, events e staticRecipients non vuote, abouts non vuota con AnyType non nullo, recipientsProvider non nullo)	Valido	O1
TF2	notification.template = null; resto nominale	Invalido	O2
TF3	notification.events = null; resto nominale	Invalido	O2
TF4	notification.staticRecipients = null; resto nominale	Invalido	O2
TF5	notification.abouts con un elemento il cui AnyType è null; resto nominale	Invalido	O2
TF6	notification.abouts = vuota; resto nominale	Valido/Limite	O3
TF7	notification.recipientsProvider = null; resto nominale	Valido/Limite	O4
TF8	notification.events = vuota (non nulla); resto nominale	Valido/Limite	O5
TF9	notification.staticRecipients = vuota (non nulla); resto nominale	Valido/Limite	O5

Tabella 18: getNotificationTO(...): legenda degli oracoli

Codice	Comportamento atteso
O1	NotificationTO popolato fedelmente da tutti i campi di notification; nessuna eccezione
O2 <i>(ipotesi)</i>	Eccezione attesa quando un sotto-campo composto di notification è nullo e viene copiato direttamente senza controllo visibile – da verificare sperimentalmente su ciascuno dei quattro sotto-campi interessati (TF2-TF5)
O3	notificationTO.abouts risulta vuota; nessuna eccezione
O4	notificationTO.recipientsProvider non valorizzato (resta null); nessuna eccezione – unico campo con un ramo esplicito distinto dalla semplice copia diretta
O5	Il campo TO corrispondente (events o staticRecipients) risulta una lista vuota; nessuna eccezione – distingue questo esito da quello del campo sorgente nullo (TF3/TF4, O2)

Nota sulla struttura: O2 copre 4 test frame (TF2-TF5) con un solo codice, perché è la stessa ipotesi (eccezione su sotto-campo composto nullo) verificata su quattro campi diversi.

Figura 12: NotificationDataBinderImplBBTest.java, metodo tf11_aboutRemovedWhenNotInTO

```

352     @Test
353     @DisplayName("TF11: about iniziale non presente nella TO -> rimosso (07, ipotesi da verificare)")
354     void tf11_aboutRemovedWhenNotInTO() {
355         // Arrange
356         Notification notification = mockNotification();
357         NotificationTO to = validNotificationTO(); // abouts vuota di default
358
359         AnyType existingType = mock(AnyType.class);
360         when(existingType.getKey()).thenReturn(t: "OTHER_TYPE");
361         AnyAbout existingAbout = mock(AnyAbout.class);
362         when(existingAbout.getAnyType()).thenReturn(existingType);
363         List<AnyAbout> abouts = new ArrayList<>(List.of(existingAbout));
364         doReturn(abouts).when(notification).getAbouts();
365
366         doReturn(Optional.of(mock(MailTemplate.class))).when(mailTemplateDAO).findById(key: "mail-template-1");
367
368         // Act
369         assertDoesNotThrow(() -> binder.update(notification, to));
370
371         // Assert: stessa istanza di lista passata al mock, mutata in place da removeIf(...)
372         assertTrue(abouts.isEmpty());
373     }
374 }

```

Figura 13: Report JaCoCo (solo test BB)

NotificationDataBinderImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
create(NotificationTO)		0%		n/a	1	1	3	3	1	1
lambda\$update\$5(NotificationTO, AnyAbout)		90%		50%	1	2	0	1	0	1
update(Notification, NotificationTO)		100%		100%	0	6	0	38	0	1
getNotificationTO(Notification)		100%		100%	0	2	0	16	0	1
lambda\$update\$3(Notification, Map.Entry, AnyType)		100%		50%	1	2	0	8	0	1
NotificationDataBinderImpl(MailTemplateDAO, AnyTypeDAO, ImplementationDAO, EntityFactory, IntAttrNameParser)		100%		n/a	0	1	0	7	0	1
lambda\$update\$0(SyncopeClientException, String)		100%		100%	0	2	0	5	0	1
lambda\$update\$2(Notification, Map.Entry)		100%		n/a	0	1	0	1	0	1
lambda\$getNotificationTO\$0(NotificationTO, AnyAbout)		100%		n/a	0	1	0	2	0	1
lambda\$update\$6(NotificationTO)		100%		n/a	0	1	0	2	0	1
lambda\$update\$4(Map.Entry)		100%		n/a	0	1	0	1	0	1
lambda\$update\$1(Map.Entry)		100%		n/a	0	1	0	1	0	1
static(...)		100%		n/a	0	1	0	1	0	1
Total	13 of 353	96%	2 of 18	88%	3	22	3	82	1	13

```

145.         // 1. add or update all (valid) abouts from TO
146.         notificationTO.getAbouts().entrySet().stream().
147.             filter(entry -> StringUtils.isNotBlank(entry.getValue()))
148.             .forEach(entry -> anyTypeDAO.findById(entry.getKey()).ifPresentOrElse(
149.                 anyType -> {
150.                     AnyAbout about = notification.getAbout(anyType).orElse(null);
151.                     if (about == null) {
152.                         about = entityFactory.newEntity(AnyAbout.class);
153.                         about.setAnyType(anyType);
154.                         about.setNotification(notification);
155.
156.                         notification.add(about);
157.                     }
158.                     about.set(entry.getValue());
159.                 },
160.                 () -> LOG.debug("Invalid AnyType {} specified, ignoring...", entry.getKey()));
161.
162.         // 2. remove all abouts not contained in the TO
163.         notification.getAbouts().
164.             removeIf(anyAbout -> !notificationTO.getAbouts().containsKey(anyAbout.getAnyType().getKey()));

```

Nota: branches missed del metodo update().

Figura 14: Report JaCoCo (BB, WB)
NotificationDataBinderImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Ctxy	Missed	Lines	Missed	Methods
• create(NotificationTO)	0%		100%	n/a	1	1	3	3	1	1
• update(Notification, NotificationTO)	100%		100%	0	6	0	38	0	1	
• getNotificationTO(Notification)	100%		100%	0	2	0	16	0	1	
• lambda\$update\$3(Notification, Map.Entry, Any.Type)	100%		100%	0	2	0	8	0	1	
• NotificationDataBinderImpl(MailTemplateDAO, AnyTypeDAO, ImplementationDAO, EntityFactory, IntAttrNameParser)	100%		n/a	0	1	0	7	0	1	
• lambda\$update\$0(SyncopeClientException, String)	100%		100%	0	2	0	5	0	1	
• lambda\$update\$2(Notification, Map.Entry)	100%		n/a	0	1	0	1	0	1	
• lambda\$update\$5(NotificationTO, Any.About)	100%		100%	0	2	0	1	0	1	
• lambda\$getNotificationTO\$0(NotificationTO, Any.About)	100%		n/a	0	1	0	2	0	1	
• lambda\$update\$6(NotificationTO)	100%		n/a	0	1	0	2	0	1	
• lambda\$update\$4(Map.Entry)	100%		n/a	0	1	0	1	0	1	
• lambda\$update\$1(Map.Entry)	100%		n/a	0	1	0	1	0	1	
• static {...}	100%		n/a	0	1	0	1	0	1	
Total	12 of 353	96%	0 of 18	100%	1	22	3	82	1	13

Figura 15: Report PIT (BB, WB)
Pit Test Coverage Report

Package Summary

org.apache.syncope.core.provisioning.java.data

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	96% 79/82	52% 23/44	55% 23/42

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
NotificationDataBinderImpl.java	96% 79/82	52% 23/44	55% 23/42

Report generated by [PIT 1.25.8](#) [support](#)

Mutations

73	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setKey → SURVIVED Covering tests
74	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setTemplate → KILLED
76	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientsFIQL → SURVIVED Covering tests
78	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientAttrName → SURVIVED Covering tests
79	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSelfAsRecipient → SURVIVED Covering tests
80	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSender → SURVIVED Covering tests
81	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSubject → SURVIVED Covering tests
82	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setTraceLevel → SURVIVED Covering tests
83	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setActive → SURVIVED Covering tests
85	1. removed call to java/util/List::forEach → KILLED
88	1. negated conditional → KILLED
89	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientsProvider → KILLED
92	1. replaced return value with null for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::getNotificationTO → KILLED
98	1. removed call to org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::update → NO_COVERAGE
99	1. replaced return value with null for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::create → NO_COVERAGE
104	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientsFIQL → SURVIVED Covering tests
106	1. removed call to java/util/List::clear → SURVIVED Covering tests
109	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientAttrName → SURVIVED Covering tests
110	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSelfAsRecipient → SURVIVED Covering tests
111	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSender → SURVIVED Covering tests
112	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSubject → SURVIVED Covering tests
113	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setTraceLevel → SURVIVED Covering tests
114	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setActive → SURVIVED Covering tests
116	1. removed call to java/util/List::clear → SURVIVED Covering tests
122	1. negated conditional → KILLED
125	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setTemplate → KILLED
127	1. negated conditional → KILLED
131	1. negated conditional → KILLED
132	1. removed call to java/util/List::forEach → KILLED
134	1. negated conditional → KILLED
141	1. negated conditional → KILLED
147	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$1 → KILLED
	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$1 → KILLED
148	1. removed call to java/util/Optional::ifPresentOrElse → KILLED
	2. removed call to java/util/stream/Stream::forEach → KILLED
151	1. negated conditional → KILLED
153	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::setAnyType → SURVIVED Covering tests
154	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::setNotification → SURVIVED Covering tests
158	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::set → KILLED
164	1. replaced boolean return with true for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$5 → KILLED
	2. negated conditional → KILLED
175	1. negated conditional → KILLED
176	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientsProvider → KILLED
178	1. removed call to java/util/Optional::ifPresentOrElse → KILLED

Figura 16: NotificationDataBinderImplMTTest.java, metodo mt3_update_clearsListsBeforeReplacing

```

184     @Test
185     @DisplayName("MT3: update sostituisce (clear+addAll) il contenuto di staticRecipients/events, non lo accoda")
186     void mt3_update_clearsListsBeforeReplacing() {
187         // Arrange
188         Notification notification = mock(Notification.class);
189         List<String> staticRecipients = new ArrayList<>(List.of( e1: "old@example.com"));
190         when(notification.getStaticRecipients()).thenReturn(staticRecipients);
191         List<String> events = new ArrayList<>(List.of( e1: "oldEvent"));
192         when(notification.getEvents()).thenReturn(events);
193
194         NotificationTO to = validNotificationTO(); // events=[userCreate], staticRecipients vuota
195
196         doReturn(Optional.of(mock(MailTemplate.class))).when(mailTemplateDAO).findById( key: "mail-template-1");
197
198         // Act
199         assertDoesNotThrow(() -> binder.update(notification, to));
200
201         // Assert: stesse istanze di lista passate al mock, mutate in place da clear()+addAll()
202         assertEquals(List.of( e1: "userCreate"), events);
203         assertEquals(List.of(), staticRecipients);
204     }

```

Figura 17: Report PIT (BB, WB, MT)

Pit Test Coverage Report

Package Summary

org.apache.syncope.core.provisioning.java.data

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	96% 79/82	95% 42/44	100% 42/42

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
NotificationDataBinderImpl.java	96% 79/82	95% 42/44	100% 42/42

Report generated by PIT 1.25.8 [support](#)

Mutations

73	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setKey → KILLED
74	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setTemplate → KILLED
76	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientsFIQL → KILLED
78	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientAttrName → KILLED
79	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSelfAsRecipient → KILLED
80	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSender → KILLED
81	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSubject → KILLED
82	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setTraceLevel → KILLED
83	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setActive → KILLED
85	1. removed call to java/util/List::forEach → KILLED
88	1. negated conditional → KILLED
89	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientsProvider → KILLED
92	1. replaced return value with null for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::getNotificationTO → KILLED
98	1. removed call to org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::update → NO_COVERAGE
99	1. replaced return value with null for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::create → NO_COVERAGE
104	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientsFIQL → KILLED
106	1. removed call to java/util/List::clear → KILLED
109	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientAttrName → KILLED
110	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSelfAsRecipient → KILLED
111	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSender → KILLED
112	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSubject → KILLED
113	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setTraceLevel → KILLED
114	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setActive → KILLED
116	1. removed call to java/util/List::clear → KILLED
122	1. negated conditional → KILLED
125	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setTemplate → KILLED
127	1. negated conditional → KILLED
131	1. negated conditional → KILLED
132	1. removed call to java/util/List::forEach → KILLED
134	1. negated conditional → KILLED
141	1. negated conditional → KILLED
147	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$1 → KILLED
147	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$1 → KILLED
148	1. removed call to java/util/Optional::ifPresentOrElse → KILLED
148	2. removed call to java/util/stream/Stream::forEach → KILLED
151	1. negated conditional → KILLED
153	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::setAnyType → KILLED
154	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::setNotification → KILLED
158	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::set → KILLED
164	1. replaced boolean return with true for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$5 → KILLED
164	2. negated conditional → KILLED
175	1. negated conditional → KILLED
176	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientsProvider → KILLED
178	1. removed call to java/util/Optional::ifPresentOrElse → KILLED

Tabella 19: Randoop su NotificationDataBinderImpl, suite curata

ID	Metodo	Input caratterizzante	Esito osservato	Causa verificata sul sorgente	Classificazione
RT 1	update(...)	NotificationTO vuoto (nessun campo impostato)	SyncopClientException, RequiredValuesMissing [template, events]	Validazione accumula tutti i campi mancanti/non validi in un'unica eccezione finale	Comportamento corretto
RT 2	update(...)	template = "hi!" (valorizzato, non risolvibile)	Stesso messaggio di RT1, incluso "template" tra i mancanti	Controllo non su stringa vuota, ma su risoluzione fallita via mailTemplateDAO.findById(...).orElse(null)	Comportamento corretto
RT 3	getNotificationTO(...)	Notification mock "nudo" (nessuno stub)	NullPointerException a NotificationDataBinderImpl.java:74	getTemplate() ritorna null solo sul mock; su entità JPA reale il campo è @JoinColumn(..., optional = false) – stato irraggiungibile con dati persistiti	Non un bug funzionale; escluso da reliability

Nota: RT1 e RT2 restano entrambe in suite nonostante l'esito osservato identico: il terzo criterio di filtraggio (Sezione 4.6) tiene un solo rappresentante per causa profonda condivisa, ma qui le cause verificate sono diverse – campo mancante vs. lookup fallito.

Esempio 4: error-revealing test di Randoop

Campo	Valore
Contesto	RT3, error-revealing test di Randoop su getNotificationTO()
Scenario minimo	Notification notification = Mockito.mock(Notification.class); (nessuno stub) binder.getNotificationTO(notification);
Esito osservato	NullPointerException a NotificationDataBinderImpl.java:74 (notification.getTemplate().getKey())
Causa isolata	getTemplate() ritorna null sul mock non stubbato – verificato via jshell
Verifica sul sorgente reale	JPA Notification.java: @JoinColumn(..., optional = false) – vincolo NOT NULL a livello di schema; lo stato non è raggiungibile su dati realmente persistiti
Stato	Non un bug funzionale; test mantenuto come documentazione dell'invariante implicita

```

70  @Override
71  public NotificationTO getNotificationTO(final Notification notification) {
72      NotificationTO notificationTO = new NotificationTO();
73      notificationTO.setKey(notification.getKey());
74      notificationTO.setTemplate(notification.getTemplate().getKey());
75      notificationTO.getEvents().addAll(notification.getEvents());
76      notificationTO.setRecipientsFIQL(notification.getRecipientsFIQL());
77      notificationTO.getStaticRecipients().addAll(notification.getStaticRecipients());
78      notificationTO.setRecipientAttrName(notification.getRecipientAttrName());
79      notificationTO.setSelfAsRecipient(notification.isSelfAsRecipient());
80      notificationTO.setSender(notification.getSender());
81      notificationTO.setSubject(notification.getSubject());
82      notificationTO.setTraceLevel(notification.getTraceLevel());
83      notificationTO.setActive(notification.isActive());
84
85      notification.getAbouts().forEach(about -> notificationTO.getAbouts().
86          put(about.getAnyType().getKey(), about.get()));
87
88      if (notification.getRecipientsProvider() != null) {
89          notificationTO.setRecipientsProvider(notification.getRecipientsProvider().getKey());
90      }
91
92      return notificationTO;
93  }

```

```

75     @Test
76     void getNotificationToWithBareMockThrowsNpeOnUnenforcedInvariant() {
77         NotificationDataBinderImpl binder = NotificationDataBinderImplRandoopSeeds.build();
78         Notification notification = NotificationDataBinderImplRandoopSeeds.notification();
79
80         assertThrows(NullPointerException.class,
81             () -> binder.getNotificationTO(notification));
82     }
83 }

```

Figura 18: Report JaCoCo (Randoop)
NotificationDataBinderImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
update(Notification, NotificationTO)		58%		40%	5	6	17	38	0	1
getNotificationTO(Notification)		11%		0%	1	2	14	16	0	1
lambda\$update\$3(Notification, Map.Entry, AnyType)		0%		0%	2	2	8	8	1	1
lambda\$update\$5(SyncopeClientException, String)		0%		0%	2	2	5	5	1	1
lambda\$update\$2(Notification, Map.Entry)		0%		n/a	1	1	1	1	1	1
create(NotificationTO)		0%		n/a	1	1	3	3	1	1
lambda\$update\$5(NotificationTO, Any>About)		0%		0%	2	2	1	1	1	1
lambda\$setNotificationTO\$0(NotificationTO, Any>About)		0%		n/a	1	1	2	2	1	1
lambda\$update\$6(NotificationTO)		0%		n/a	1	1	2	2	1	1
lambda\$update\$4(Map.Entry)		0%		n/a	1	1	1	1	1	1
lambda\$update\$1(Map.Entry)		0%		n/a	1	1	1	1	1	1
NotificationDataBinderImpl(MailTemplateDAO, AnyTypeDAO, ImplementationDAO, EntityFactory, InAttrNameParser)		100%		n/a	0	1	0	7	0	1
static(...)		100%		n/a	0	1	0	1	0	1
Total	237 of 353	32%	14 of 18	22%	18	22	51	82	9	13

```

70.     @Override
71.     public NotificationTO getNotificationTO(final Notification notification) {
72.         NotificationTO notificationTO = new NotificationTO();
73.         notificationTO.setKey(notification.getKey());
74.         notificationTO.setTemplate(notification.getTemplate().getKey());
75.         notificationTO.getEvents().addAll(notification.getEvents());
76.         notificationTO.setRecipientsFIQL(notification.getRecipientsFIQL());
77.         notificationTO.getStaticRecipients().addAll(notification.getStaticRecipients());
78.         notificationTO.setRecipientAttrName(notification.getRecipientAttrName());
79.         notificationTO.setSelfAsRecipient(notification.isSelfAsRecipient());
80.         notificationTO.setSender(notification.getSender());
81.         notificationTO.setSubject(notification.getSubject());
82.         notificationTO.setTraceLevel(notification.getTraceLevel());
83.         notificationTO.setActive(notification.isActive());
84.
85.         notification.getAbouts().forEach(about -> notificationTO.getAbouts().
86.             put(about.getAnyType().getKey(), about.get()));
87.
88.         if (notification.getRecipientsProvider() != null) {
89.             notificationTO.setRecipientsProvider(notification.getRecipientsProvider().getKey());
90.         }
91.         return notificationTO;
92.     }
93.
102.    @Override
103.    public void update(final Notification notification, final NotificationTO notificationTO) {
104.        notification.setRecipientsFIQL(notificationTO.getRecipientsFIQL());
105.
106.        notification.getStaticRecipients().clear();
107.        notification.getStaticRecipients().addAll(notificationTO.getStaticRecipients());
108.
109.        notification.setRecipientAttrName(notificationTO.getRecipientAttrName());
110.        notification.setSelfAsRecipient(notificationTO.isSelfAsRecipient());
111.        notification.setSender(notificationTO.getSender());
112.        notification.setSubject(notificationTO.getSubject());
113.        notification.setTraceLevel(notificationTO.getTraceLevel());
114.        notification.setActive(notificationTO.isActive());
115.
116.        notification.getEvents().clear();
117.        notification.getEvents().addAll(notificationTO.getEvents());
118.
119.        SyncopeClientException sce = SyncopeClientException.build(ClientExceptionType.RequiredValuesMissing);
120.
121.        MailTemplate template = mailTemplateDAO.findById(notificationTO.getTemplate().orElse(null));
122.        if (template == null) {
123.            sce.getElements().add("template");
124.        }
125.        notification.setTemplate(template);
126.
127.        if (notification.getEvents().isEmpty()) {
128.            sce.getElements().add("events");
129.        }
130.
131.        if (!notification.getStaticRecipients().isEmpty()) {
132.            notification.getStaticRecipients().forEach(mail -> {
133.                Matcher matcher = Entity.EMAIL_PATTERN.matcher(mail);
134.                if (!matcher.matches()) {
135.                    LOG.error("Invalid mail address: {}", mail);
136.                    sce.getElements().add("staticRecipients: " + mail);
137.                }
138.            });
139.        }

```

Figura 19: Report PIT (Randoop)

Pit Test Coverage Report

Package Summary

org.apache.syncope.core.provisioning.java.data

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	39% 32/82	7% 3/44	20% 3/15

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
NotificationDataBinderImpl.java	39% 32/82	7% 3/44	20% 3/15

Report generated by PIT 1.25.8 [support](#)

Mutations

73	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setKey → SURVIVED Covering tests
74	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setTemplate → NO_COVERAGE
76	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientsFIQL → NO_COVERAGE
78	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientAttrName → NO_COVERAGE
79	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSelfAsRecipient → NO_COVERAGE
80	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSender → NO_COVERAGE
81	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setSubject → NO_COVERAGE
82	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setTraceLevel → NO_COVERAGE
83	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setActive → NO_COVERAGE
85	1. removed call to java/util/List::forEach → NO_COVERAGE
88	1. negated conditional → NO_COVERAGE
89	1. removed call to org/apache/syncope/common/lib/to/NotificationTO::setRecipientsProvider → NO_COVERAGE
92	1. replaced return value with null for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::getNotificationTO → NO_COVERAGE
98	1. removed call to org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::update → NO_COVERAGE
99	1. replaced return value with null for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::create → NO_COVERAGE
104	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientsFIQL → SURVIVED Covering tests
106	1. removed call to java/util/List::clear → SURVIVED Covering tests
109	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientAttrName → SURVIVED Covering tests
110	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSelfAsRecipient → SURVIVED Covering tests
111	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSender → SURVIVED Covering tests
112	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setSubject → SURVIVED Covering tests
113	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setTraceLevel → SURVIVED Covering tests
114	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setActive → SURVIVED Covering tests
116	1. removed call to java/util/List::clear → SURVIVED Covering tests
122	1. negated conditional → KILLED
125	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setTemplate → SURVIVED Covering tests
127	1. negated conditional → KILLED
131	1. negated conditional → SURVIVED Covering tests
132	1. removed call to java/util/List::forEach → NO_COVERAGE
134	1. negated conditional → NO_COVERAGE
141	1. negated conditional → KILLED
147	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$1 → NO_COVERAGE
	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$1 → NO_COVERAGE
148	1. removed call to java/util/Optional::ifPresentOrElse → NO_COVERAGE
	2. removed call to java/util/stream/Stream::forEach → NO_COVERAGE
151	1. negated conditional → NO_COVERAGE
153	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::setAnyType → NO_COVERAGE
154	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::setNotification → NO_COVERAGE
158	1. removed call to org/apache/syncope/core/persistence/api/entity/AnyAbout::set → NO_COVERAGE
164	1. replaced boolean return with true for org/apache/syncope/core/provisioning/java/data/NotificationDataBinderImpl::lambda\$update\$5 → NO_COVERAGE
	2. negated conditional → NO_COVERAGE
175	1. negated conditional → NO_COVERAGE
176	1. removed call to org/apache/syncope/core/persistence/api/entity/Notification::setRecipientsProvider → NO_COVERAGE
178	1. removed call to java/util/Optional::ifPresentOrElse → NO_COVERAGE

Prompt 4: LLM Zero-shot

As a professional software tester who writes Java test methods, generate a complete JUnit 5 test file (NotificationDataBinderImplLLMZeroShotTest.java) to test the following methods of the class named NotificationDataBinderImpl:

- update(Notification notification, NotificationTO notificationTO)
- getNotificationTO(Notification notification)

Your output file must start with `###Test START##` and finish with `###Test END##`.

Prompt 5: LLM Few-shot

As a professional software tester who writes Java test methods, consider the following examples:

```
@Test
@DisplayName("TF1: scenario nominale, tutti i campi validi -> nessuna eccezione (O1)")
void tf1_nominal() {
    // Arrange
    Notification notification = mockNotification();

    NotificationTO to = validNotificationTO();
    to.getStaticRecipients().add("user@example.com");
    to.getAbouts().put("USER", "cond==true");
    to.setRecipientsProvider("provider-1");

    MailTemplate template = mock(MailTemplate.class);
    doReturn(Optional.of(template)).when(mailTemplateDAO).findById("mail-template-1");

    AnyType anyType = mock(AnyType.class);
    doReturn(Optional.of(anyType)).when(anyTypeDAO).findById("USER");
    doReturn(Optional.empty()).when(notification).getAbout(eq(anyType));
    AnyAbout newAbout = mock(AnyAbout.class);
    when(entityFactory.newEntity(AnyAbout.class)).thenReturn(newAbout);
    Implementation provider = mock(Implementation.class);
    doReturn(Optional.of(provider)).when(implementationDAO).findById("provider-1");
    // intAttrNameParser.parse(...) non stubbato -> comportamento di default (nessuna eccezione)
    // Act
    assertDoesNotThrow(() -> binder.update(notification, to));
    // Assert
    verify(notification).setTemplate(template);
    verify(notification).add(newAbout);
    verify(notification).setRecipientsProvider(provider);
}

@Test
@DisplayName("TF2: notification nulla -> NullPointerException (O2)")
void tf2_notificationNull() {
    // Arrange
    NotificationTO to = validNotificationTO();
    // Act & Assert
    assertThrows(NullPointerException.class, () -> binder.update(null, to));
}
```

Now, generate a complete JUnit 5 test file (NotificationDataBinderImplLLMFewShotTest.java) to test the following methods of the class named NotificationDataBinderImpl:

- update(Notification notification, NotificationTO notificationTO)
- getNotificationTO(Notification notification)

Prompt 6: LLM Guided ToT

Imagine three different experts in software testing who are tasked with developing comprehensive JUnit 5 test cases for the following Java class. To comprehensively test the following methods of the class named NotificationDataBinderImpl they must following these steps:

- search(String keyword, Pageable pageable)
- run(CommandTO command)

they must following these steps:

- Extract and list the signatures of the two methods above
- For each method, generate a basic JUnit 5 test case that checks the method's functionality
- Given the source code of the class and the listed methods, identify potential edge cases and exception handling scenarios that should be tested
- Generate JUnit 5 test cases that specifically test for the identified edge cases and exceptions
- Merge all the individual test cases into a complete JUnit 5 test file NotificationDataBinderImplLLMToTTest.java

All experts will propose one test cases for each method, share it with the group, and then proceed to the next step. If any expert realizes they're wrong at any point, they leave.

Figura 20: NotificationDataBinderImplLLMZeroShotRaw.java, evidenza delle due cause di errore di compilazione

```
68     @Test
69     void getNotificationTOCopiesAllNotificationFields() {
70         Notification notification = mock(Notification.class);
71
72         MailTemplate template = mock(MailTemplate.class);
73         when(template.getKey()).thenReturn("welcomeTemplate");
74
75         Implementation recipientsProvider = mock(Implementation.class);
76         when(recipientsProvider.getKey()).thenReturn("recipientsProviderImpl");
77
78         AnyType userType = mock(AnyType.class);
79         when(userType.getKey()).thenReturn("USER");
80
81         AnyAbout about = mock(AnyAbout.class);
82         when(about.getAnyType()).thenReturn(userType);
83         when(about.get()).thenReturn("username==john");
```

Nota: Causa A – `when(...).thenReturn(...)` su DAO con tipo di ritorno a bound wildcard

```
89     Set<String> staticRecipients = new LinkedHashSet<>();
90     staticRecipients.add("admin@example.com");
91     staticRecipients.add("audit@example.com");
92
93     Set<AnyAbout> abouts = new LinkedHashSet<>();
94     abouts.add(about);
95
96     when(notification.getKey()).thenReturn("notificationKey");
97     when(notification.getTemplate()).thenReturn(template);
98     when(notification.getEvents()).thenReturn(events);
99     when(notification.getRecipientsFIQL()).thenReturn("username==john");
100    when(notification.getStaticRecipients()).thenReturn(staticRecipients);
101    when(notification.getRecipientAttrName()).thenReturn("email");
102    when(notification.isSelfAsRecipient()).thenReturn(true);
103    when(notification.getSender()).thenReturn("sender@example.com");
104    when(notification.getSubject()).thenReturn("Subject");
105    when(notification.getTraceLevel()).thenReturn(TraceLevel.ALL);
106    when(notification.isActive()).thenReturn(true);
107    when(notification.getAbouts()).thenReturn(abouts);
108    when(notification.getRecipientsProvider()).thenReturn(recipientsProvider);
```

Nota: Causa B – `Set/LinkedHashSet` al posto di `List` sui getter di Notification

Figura 21: NotificationDataBinderImplLLMFewShotRaw.java, evidenza dei due errori di compilazione

```
251     @Test
252     @DisplayName("update: about non presente nel T0 viene rimosso dalla notification")
253     void updateRemovesAboutsNotContainedInT0() {
254         // Arrange
255         Notification notification = mockNotification("email");
256
257         org.apache.syncope.core.persistence.api.entity.AnyType anyType =
258             mock(org.apache.syncope.core.persistence.api.entity.AnyType.class);
259         when(anyType.getKey()).thenReturn("USER");
260
261         AnyAbout existingAbout = mock(AnyAbout.class);
262         when(existingAbout.getAnyType()).thenReturn(anyType);
263
264         notification.getAbouts().add(existingAbout);
265
266         NotificationT0 to = validNotificationT0();
267
268         MailTemplate template = mock(MailTemplate.class);
269         doReturn(Optional.of(template)).when(mailTemplateDAO).findById("tpl-1");
270
271         // Act
272         assertDoesNotThrow(() -> binder.update(notification, to));
273
274         // Assert
275         assertFalse(notification.getAbouts().contains(existingAbout));
276     }
```

Nota: getAbouts() è dichiarato List<? extends AnyAbout>: il compilatore non permette .add(...) su un wildcard ? extends, perché non può garantire che existingAbout sia del sottotipo esatto catturato in quel punto, indipendentemente da cosa contenga la variabile a runtime.

Figura 22: NotificationDataBinderImplLLMToTRaw.java, evidenza delle tre cause di errore e della deviazione di scope

```
149     @Test
150     void createShouldCreateAndPopulateNotification() throws Exception {
151         Notification notification = mockMutableNotification();
152         MailTemplate template = mock(MailTemplate.class);
153
154         NotificationT0 notificationT0 = validNotificationT0();
155
156         when(entityFactory.newEntity(Notification.class)).thenReturn(notification);
157         when(mailTemplateDAO.findById("template-key")).thenReturn(Optional.of(template));
158
159         Object parsedAttrName = mock(Object.class);
160         when(intAttrNameParser.parse("email", AnyTypeKind.USER)).thenReturn(parsedAttrName);
```

Causa A (10 occorrenze): when(...).thenReturn(...) su DAO a bound wildcard.

Causa B (1 occorrenza, nuova): intAttrNameParser.parse(...) è dichiarato per restituire IntAttrName, non Object – il mock (mock(Object.class)) e il tipo della variabile sono entrambi sbagliati.

```
73     @Test
74     void getNotificationT0ShouldMapNotificationFields() {
75         Notification notification = mock(Notification.class);
76         MailTemplate template = mock(MailTemplate.class);
77         Implementation recipientsProvider = mock(Implementation.class);
78
79         AnyType anyType = mock(AnyType.class);
80         AnyAbout about = mock(AnyAbout.class);
81
82         Set<String> events = new LinkedHashSet<>();
83         events.add("user:create");
84
85         Set<String> staticRecipients = new LinkedHashSet<>();
86         staticRecipients.add("admin@example.com");
87
88         Set<AnyAbout> abouts = new LinkedHashSet<>();
89         abouts.add(about);
```

Causa C (9 occorrenze, helper mockMutableNotification): Set/LinkedHashSet invece di List.

Tabella 20: LLM su NotificationDataBinderImpl, coverage per tecnica

Tecnica	Istruzioni	Branch
Zero-shot	96%	100%
Few-shot	96%	94%
ToT	96%	77%

Tabella 21: (NotificationDataBinderImpl) test forniti in input all'LLM per la generazione delle varianti, se passano ancora i test della classe C0

Classe	Versione	Test forniti come vincolo	Compila	Test manuali	Test Randoop	Test LLM
C0	Originale	— (baseline)	Sì	24/24	3/3	34/34
C1	Refactored	Nessuno	Sì	24/24	3/3	34/34
C2	Refactored	BB	Sì	24/24	3/3	34/34
C3	Refactored	BB + CF (JaCoCo/WB)	Sì	24/24	3/3	34/34
C4	Refactored	BB + CF + MT	Sì	24/24	3/3	34/34

Nota: vale C1=C2=C3=C4 al livello del byte-code.

Tabella 22: NotificationDataBinderImpl, C0 vs. C1

Suite	Istruzioni	Δ	Branch	Δ	Line Cov. (PIT)	Δ	Mutation Cov.	Δ	Test Strength	Δ
Randoop (curato)	32%→33%	+1%	22%→22%	0%	39%→39%	0%	7% (3/44)→2% (1/44)	-5%	20%→7%	-13%
Zero-shot (singola)	96%→94%	-2%	100%→89%	-11%	100%→94%	-6%	89% (39/44)→86% (38/44)	-3%	93% (39/42)→90%	-3%
ToT (singola)	96%→76%	-20%	77%→56%	-21%	100%→78%	-22%	89% (39/44)→73% (32/44)	-16%	93% (39/42)→86%	-7%

Appendice: Prompt LLM per generare le classi varianti C1-C4

Il file sorgente della classe originale viene messo in allegato.

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_1 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicata>.

No test suite is provided as input for this variant (C_1); you must rely only on the class semantics and the smell description above to guide the refactoring.

Do not include in C_1 changes different from what I asked. In fact, consider that C_1 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

Prompt 7: per generare la variante C1

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_2 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicato>.

Make sure C_2 passes the attached Black Box test suite (<nome_classe>BBTest.java), which was designed with Category Partition and Boundary Value Analysis and is attached as a separate file. These tests must continue to pass unchanged against the refactored class.

Do not include in C_2 changes different from what I asked. In fact, consider that C_2 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

Prompt 8: per generare la variante C2, in allegato anche i test BB

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_3 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicato>.

Make sure C_3 passes the attached Black Box test suite (<nome_classe>BBTest.java) and the attached Control Flow test suite (<nome_classe>WBTest.java, developed to improve statement and branch coverage using JaCoCo). Both suites are attached as separate files and must continue to pass unchanged against the refactored class.

Do not include in C_3 changes different from what I asked. In fact, consider that C_3 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

Prompt 9: per generare la variante C3, in allegato anche i test BB e WB

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_4 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicato>.

Make sure C_4 passes the attached Black Box test suite (<nome_classe>BBTest.java), the attached Control Flow test suite (<nome_classe>WBTest.java, developed to improve statement and branch coverage using JaCoCo), and the attached Mutation test suite (<nome_classe>MTTest.java, developed to improve mutation score using PIT). All three suites are attached as separate files and must continue to pass unchanged against the refactored class.

Do not include in C_4 changes different from what I asked. In fact, consider that C_4 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

C_0 source code: <codice_sorgente>

Prompt 10: per generare la variante C4, in allegato i test BB, WB e MT. Il codice sorgente viene incollato in prompt per il limite di Copilot che supporta massimo tre file in allegato.